

Um guia prático do zero ao avançado

DevOps e GitOps com ArgoCD

Volume I - Nível Básico

Sumário

Capítulo 1 — Introdução

Capítulo 2 — Introdução ao DevOps

Capítulo 3 — Arquitetura de Microserviços

Capítulo 4 — Introdução ao GitOps

Capítulo 5 — GitOps com ArgoCD

Capítulo 6 — Preparação do Ambiente

Capítulo 7 — Instalação e Acesso ao ArgoCD

Capítulo 8 — Uso Básico do ArgoCD

Capítulo 9 — Revision e Sync Policy

Capítulo 10 — Projects, Namespaces e Segregação de Ambientes

Capítulo 11 — ApplicationSet

Capítulo 12 — Sync Window

Capítulo 13 — Ignore Differences

Capítulo 14 — Gestão de Acesso e RBAC no ArgoCD

Conclusão

Referências

Apêndice A — Instalação do Rancher Desktop

Capítulo 1 — Introdução

DevOps e GitOps com ArgoCD

Um guia prático do zero ao avançado

1.1 Contexto e Motivação

Nos últimos anos, a forma como aplicações são desenvolvidas, entregues e operadas passou por uma transformação profunda. A adoção de arquiteturas baseadas em microsserviços, aliada à popularização do Kubernetes como plataforma padrão de orquestração de containers, trouxe ganhos significativos de escalabilidade, resiliência e velocidade de entrega.

Por outro lado, essa evolução também aumentou drasticamente a complexidade operacional. Ambientes compostos por dezenas ou centenas de serviços exigem processos confiáveis, auditáveis e automatizados para garantir consistência entre código, configuração e infraestrutura.

Nesse cenário, práticas modernas como **DevOps** e **GitOps** deixaram de ser tendências e passaram a ser **requisitos fundamentais** para equipes que operam ambientes Kubernetes em produção. O Git, que antes era utilizado apenas para versionamento de código, passou a assumir um papel central também na gestão da infraestrutura e das aplicações.

É exatamente nesse ponto que o **ArgoCD** se destaca como uma das ferramentas mais importantes do ecossistema Kubernetes moderno.

1.2 Objetivo deste Ebook

Este ebook tem como objetivo apresentar, de forma **progressiva, prática e estruturada**, os conceitos e ferramentas necessários para implementar **GitOps com ArgoCD** em ambientes Kubernetes.

Ao longo dos capítulos, o leitor aprenderá:

- Os fundamentos de DevOps e GitOps
- Os desafios de ambientes baseados em microsserviços
- Como o ArgoCD funciona e qual o seu papel no modelo GitOps
- Como criar, gerenciar e sincronizar Applications
- Como utilizar manifests Kubernetes, Helm e Kustomize

- Como aplicar segregação e governança com Projects e Namespaces
- Como controlar deploys por meio de Revision, Sync Policy e Sync Window

O foco do material é **operacional e prático**, sempre conectando conceitos à sua aplicação real no dia a dia.

1.3 Público-Alvo

Este ebook é direcionado a:

- Profissionais de **DevOps, SRE e Platform Engineering**
- Desenvolvedores que atuam em ambientes Kubernetes
- Engenheiros de infraestrutura e cloud
- Estudantes de tecnologia com base em containers e Linux

Embora os conceitos sejam apresentados de forma didática, **não é um material introdutório absoluto**. Espera-se que o leitor possua ao menos conhecimentos básicos sobre:

- Containers e Docker
- Conceitos gerais de infraestrutura
- Linha de comando em ambientes Linux

Leitores que já saibam realizar o build e execução de containers terão um aproveitamento ainda maior.

1.4 O Autor

Graduado em Sistemas de Informação em 2009 pela FAESA/ES, com pós-graduação em Engenharia de Sistemas pela ESAB/ES e em Gestão de Projetos pela FAVENI/ES, o autor possui mais de 15 anos de experiência no mercado de tecnologia. Atuou como instrutor de cursos na área de tecnologia pelo SENAC/ES e atualmente trabalha como engenheiro de plataforma especialista, com forte atuação em ambientes corporativos e tecnologias modernas.

1.5 O Que Você Vai Aprender ao Longo do Livro

Ao final deste ebook, o leitor será capaz de:

- Operar o ArgoCD de forma segura e consistente

- Criar Applications declarativas e reprodutíveis
- Implementar GitOps em ambientes Kubernetes reais
- Evitar deploys manuais e inconsistências entre ambientes
- Estruturar repositórios Git para GitOps
- Controlar quando e como deploys acontecem

Mais do que aprender uma ferramenta, o objetivo é **internalizar um modelo mental moderno de entrega e operação de software**.

1.6 Estrutura do Conteúdo

O ebook está organizado em blocos progressivos:

- **Fundamentos:** DevOps, microsserviços e GitOps
- **Preparação do ambiente:** Kubernetes local e ferramentas
- **Uso do ArgoCD:** conceitos, Applications e sincronização
- **GitOps na prática:** manifests, Helm e Kustomize
- **Governança básica:** Projects, Namespaces e Sync Window

Essa organização permite que o leitor avance de forma natural, construindo conhecimento sólido antes de entrar em temas mais operacionais.

Objetivo do Capítulo (Bloco Didático)

Ao final deste capítulo, você deve ser capaz de:

- Entender o contexto que levou à adoção de GitOps
 - Compreender a proposta do ebook
 - Identificar se o conteúdo está alinhado ao seu perfil profissional
-

Checklist de Aprendizado

- Entendi o propósito do ebook
 - Sei qual problema o ArgoCD resolve
 - Compreendo para quem este material é indicado
-

Perguntas de Revisão

1. Quais fatores aumentaram a complexidade operacional em ambientes modernos?
2. Por que o Git passou a ter um papel central além do versionamento de código?

3. Qual é o foco principal deste ebook: ferramenta ou modelo operacional?



Próximo Capítulo

No próximo capítulo, vamos explorar os **fundamentos de DevOps**, entendendo como essa cultura surgiu, quais problemas ela resolve e como ela se conecta diretamente ao GitOps e ao ArgoCD.

Capítulo 2 — Introdução ao DevOps

2.1 O Surgimento do DevOps

Durante muitos anos, o desenvolvimento e a operação de software foram tratados como atividades distintas e, muitas vezes, conflitantes. Enquanto equipes de desenvolvimento tinham como objetivo entregar novas funcionalidades com rapidez, equipes de operações priorizavam estabilidade, previsibilidade e redução de riscos em produção.

Esse desalinhamento resultava em ciclos de entrega lentos, ambientes instáveis e dificuldades na resolução de incidentes. A crescente demanda por software mais ágil, escalável e disponível expôs as limitações desse modelo tradicional.

O **DevOps** surge como uma resposta a esse problema estrutural. Mais do que um conjunto de ferramentas, DevOps representa uma **mudança cultural**, que busca integrar desenvolvimento e operações em torno de objetivos comuns: entrega contínua de valor, confiabilidade e automação.

2.2 O Que É DevOps

DevOps pode ser definido como um conjunto de **práticas, princípios e valores** que promovem a colaboração entre desenvolvimento (Dev) e operações (Ops), com forte apoio de automação ao longo de todo o ciclo de vida do software.

Entre seus principais objetivos estão:

- Reduzir o tempo entre o desenvolvimento e a entrega em produção
- Aumentar a frequência de deploys
- Melhorar a qualidade e confiabilidade das aplicações
- Reduzir falhas causadas por processos manuais

É importante destacar que DevOps **não é uma ferramenta** e **não é um cargo**, mas sim uma abordagem que orienta decisões técnicas, organizacionais e operacionais.

2.3 Princípios Fundamentais do DevOps

2.3.1 Colaboração e Comunicação

Um dos pilares do DevOps é a quebra de silos entre equipes. Desenvolvimento e operações passam a compartilhar responsabilidades sobre o ciclo completo da aplicação, desde o código até a operação em produção.

Essa colaboração reduz retrabalho, melhora a previsibilidade e facilita a resolução de problemas, pois todos têm visibilidade sobre o que está sendo entregue e operado.

2.3.2 Automação

A automação é um elemento central no DevOps. Processos manuais, repetitivos e sujeitos a erro são substituídos por pipelines automatizados e definições declarativas.

Entre os processos mais comuns de automação estão:

- Build de aplicações
- Execução de testes
- Criação de artefatos
- Provisionamento de infraestrutura
- Deploy em ambientes

Ferramentas como pipelines de CI, scripts de infraestrutura como código e plataformas de orquestração são viabilizadoras desse princípio.

2.3.3 Integração Contínua e Entrega Contínua (CI/CD)

A **Integração Contínua (CI)** consiste em integrar frequentemente o código em um repositório central, validando as mudanças por meio de testes automatizados.

A **Entrega Contínua (CD)** garante que o software esteja sempre em um estado pronto para ser implantado, permitindo deploys frequentes e controlados.

Esse modelo reduz o tamanho das mudanças, facilita o rollback e melhora a confiabilidade dos ambientes.

2.3.4 Monitoramento e Feedback Contínuo

DevOps também envolve a criação de ciclos rápidos de feedback. Monitoramento, métricas e logs permitem identificar problemas rapidamente e retroalimentar o processo de desenvolvimento com informações reais do ambiente de produção.

Esse feedback contínuo é essencial para a evolução das aplicações e da própria plataforma.

2.4 Benefícios da Adoção de DevOps

A adoção consistente de práticas DevOps traz benefícios claros para organizações e equipes:

- **Entrega mais rápida** de novas funcionalidades
- **Redução de falhas** em produção
- **Maior previsibilidade** dos deploys
- **Melhor colaboração** entre times
- **Escalabilidade operacional**

Esses benefícios tornam-se ainda mais evidentes quando DevOps é combinado com plataformas como Kubernetes e práticas como GitOps.

2.5 Desafios na Adoção de DevOps

Apesar dos benefícios, a adoção de DevOps não é trivial. Entre os principais desafios estão:

- Mudança cultural e resistência organizacional
- Falta de padronização entre equipes
- Dependência excessiva de processos manuais
- Ambientes complexos e pouco observáveis

Esses desafios reforçam a necessidade de **modelos claros, automatizados e auditáveis**, especialmente em ambientes Kubernetes.

2.6 DevOps, Kubernetes e o Caminho para o GitOps

Com a popularização do Kubernetes, a complexidade operacional aumentou significativamente. Gerenciar aplicações distribuídas, múltiplos ambientes e dezenas de serviços exige mais do que pipelines tradicionais.

Nesse contexto, o DevOps evolui naturalmente para modelos mais declarativos e automatizados, abrindo espaço para práticas como **GitOps**, que veremos nos próximos capítulos.



Objetivo do Capítulo (Bloco Didático)

Ao final deste capítulo, você deve ser capaz de:

- Explicar o conceito de DevOps
 - Identificar seus principais princípios
 - Compreender os benefícios e desafios da adoção
 - Entender como DevOps se conecta ao Kubernetes
-



Exercício de Reflexão (Opcional)

Refleta sobre o ambiente em que você atua hoje:

- O deploy é manual ou automatizado?
- Existe separação clara entre desenvolvimento e operação?
- Há consistência entre ambientes (dev, hml, prod)?

Essas respostas ajudarão a contextualizar os próximos capítulos.



Checklist de Aprendizado

- Sei explicar o que é DevOps
 - Entendo por que automação é fundamental
 - Consigo relacionar DevOps com Kubernetes
-



Perguntas de Revisão

1. Por que o modelo tradicional de desenvolvimento e operação se tornou insuficiente?
 2. Qual a diferença entre CI e CD?
 3. Como o Kubernetes influenciou a evolução das práticas DevOps?
-



Próximo Capítulo

No próximo capítulo, vamos avançar para a **arquitetura de microsserviços**, entendendo como ela se relaciona com DevOps, Kubernetes e os desafios que levaram à adoção do GitOps.

Capítulo 3 — Arquitetura de Microserviços

3.1 Evolução das Arquiteturas de Software

Historicamente, muitas aplicações foram construídas seguindo o modelo **monolítico**, no qual todas as funcionalidades são implementadas, implantadas e executadas como uma única unidade. Esse modelo simplifica o início do desenvolvimento, mas tende a se tornar difícil de manter à medida que a aplicação cresce.

Com o aumento da demanda por escalabilidade, alta disponibilidade e ciclos de entrega mais rápidos, surgiram limitações claras no modelo monolítico. Alterações simples passaram a exigir deploys completos, falhas em um componente impactavam toda a aplicação e a evolução tecnológica tornava-se mais lenta.

A **arquitetura de microserviços** surge como uma resposta a esses desafios.

3.2 O Que São Microserviços

Microserviços são uma abordagem arquitetural na qual uma aplicação é composta por **múltiplos serviços pequenos, independentes e fracamente acoplados**, cada um responsável por uma funcionalidade específica do domínio de negócio.

Cada microserviço:

- Possui seu próprio ciclo de vida
- Pode ser desenvolvido e implantado de forma independente
- Comunica-se com outros serviços por meio de APIs
- Pode utilizar tecnologias diferentes, quando necessário

Essa independência permite maior flexibilidade, escalabilidade e agilidade no desenvolvimento e na operação.

3.3 Princípios Fundamentais dos Microserviços

3.3.1 Descentralização

Em arquiteturas de microserviços, decisões técnicas e operacionais são descentralizadas. Cada time pode escolher ferramentas, linguagens e estratégias mais adequadas ao seu serviço, desde que respeite contratos bem definidos de comunicação.

Essa descentralização reduz gargalos organizacionais e acelera a evolução do sistema como um todo.

3.3.2 Independência e Isolamento

Cada microsserviço deve ser isolado o suficiente para que falhas não se propaguem pelo sistema. Um problema em um serviço não deve comprometer o funcionamento dos demais.

Além disso, essa independência permite escalar apenas os serviços que realmente demandam mais recursos, otimizando custos e desempenho.

3.3.3 Comunicação por APIs

Microsserviços se comunicam por meio de APIs bem definidas, geralmente utilizando protocolos leves como HTTP/REST ou gRPC.

Essa abordagem favorece a interoperabilidade, a evolução independente dos serviços e a integração com sistemas externos.

3.4 Benefícios da Arquitetura de Microsserviços

A adoção de microsserviços traz diversos benefícios quando bem implementada:

- **Escalabilidade independente** de serviços
- **Maior velocidade de entrega**, com deploys menores e mais frequentes
- **Resiliência**, com isolamento de falhas
- **Flexibilidade tecnológica**
- **Facilidade de manutenção** em sistemas complexos

Esses benefícios se alinham diretamente com os princípios do DevOps.

3.5 Desafios dos Microsserviços

Apesar das vantagens, a arquitetura de microsserviços também introduz novos desafios, principalmente no aspecto operacional.

3.5.1 Complexidade Operacional

Gerenciar dezenas ou centenas de serviços distribuídos exige ferramentas e processos maduros. Deploys, observabilidade, segurança e configuração tornam-se mais complexos do que em sistemas monolíticos.

3.5.2 Gerenciamento de Configuração

Cada microsserviço possui sua própria configuração, variando por ambiente (desenvolvimento, homologação, produção). Manter consistência e rastreabilidade dessas configurações é um desafio significativo.

3.5.3 Consistência e Comunicação

Falhas de rede, latência e versionamento de APIs precisam ser tratados de forma explícita. Estratégias de retry, circuit breaker e observabilidade tornam-se essenciais.

3.6 Microsserviços e Kubernetes

O Kubernetes se tornou a plataforma padrão para execução de microsserviços justamente por endereçar muitos desses desafios:

- Orquestração e escalabilidade automática
- Isolamento de workloads
- Service discovery
- Gerenciamento declarativo de recursos
- Observabilidade e automação

No entanto, embora o Kubernetes resolva o **como executar**, ele não resolve completamente o **como operar** aplicações em escala.

3.7 O Problema Operacional em Ambientes de Microsserviços

Em ambientes com múltiplos microsserviços e múltiplos ambientes, surgem problemas recorrentes:

- Diferença entre o que está versionado e o que está rodando
- Deploys manuais e não rastreáveis
- Configurações divergentes entre ambientes
- Dificuldade de auditoria e rollback

Esses problemas reforçam a necessidade de um modelo operacional mais robusto e declarativo, abrindo caminho para o **GitOps**, que será explorado no próximo capítulo.

Objetivo do Capítulo (Bloco Didático)

Ao final deste capítulo, você deve ser capaz de:

- Explicar o conceito de microsserviços
 - Identificar benefícios e desafios dessa arquitetura
 - Compreender por que microsserviços aumentam a complexidade operacional
 - Relacionar microsserviços com Kubernetes e DevOps
-

Exercício de Reflexão (Opcional)

Pense em uma aplicação que você conhece ou trabalha atualmente:

- Ela é monolítica ou baseada em microsserviços?
- Quantos serviços compõem o sistema?
- Como os deploys e configurações são gerenciados hoje?

Essa reflexão ajuda a contextualizar os desafios que o GitOps se propõe a resolver.

Checklist de Aprendizado

- Sei definir o que são microsserviços
 - Entendo os benefícios dessa arquitetura
 - Reconheço os desafios operacionais envolvidos
 - Consigo relacionar microsserviços com Kubernetes
-

Perguntas de Revisão

1. Quais limitações do modelo monolítico levaram ao surgimento dos microsserviços?
 2. Por que a independência dos serviços é tão importante?
 3. Quais desafios operacionais surgem em ambientes distribuídos?
-

Próximo Capítulo

No próximo capítulo, vamos introduzir o **GitOps**, entendendo como esse modelo operacional utiliza o Git como fonte da verdade para resolver muitos dos desafios apresentados neste capítulo.

Capítulo 4 — Introdução ao GitOps

4.1 O Problema que o GitOps se Propõe a Resolver

Nos capítulos anteriores, vimos que a adoção de microsserviços e Kubernetes trouxe ganhos importantes, mas também aumentou significativamente a complexidade operacional. Em muitos ambientes, ainda é comum encontrar cenários como:

- Deploys realizados manualmente via linha de comando
- Diferenças entre ambientes de desenvolvimento, homologação e produção
- Falta de rastreabilidade sobre quem alterou o quê e quando
- Dificuldade de reproduzir ambientes ou realizar rollback confiável

Esses problemas normalmente não estão ligados à falta de ferramentas, mas sim à **ausência de um modelo operacional claro e consistente**.

O **GitOps** surge exatamente para atacar esse ponto.

4.2 O Que É GitOps

GitOps é uma prática operacional que utiliza o **Git como a única fonte da verdade** para definir o estado desejado das aplicações e da infraestrutura.

Nesse modelo:

- O estado desejado é descrito de forma declarativa
- As definições ficam versionadas em repositórios Git
- Qualquer mudança passa obrigatoriamente por commit e revisão
- Um sistema automatizado garante que o ambiente reflita o estado do Git

Em vez de executar comandos diretamente no cluster, os operadores passam a **alterar código**, e a plataforma se encarrega de aplicar essas mudanças de forma controlada.

4.3 Princípios Fundamentais do GitOps

4.3.1 Git como Fonte Única da Verdade

No GitOps, o Git deixa de ser apenas um repositório de código-fonte e passa a representar o **estado desejado completo do sistema**.

Isso inclui:

- Manifests Kubernetes
- Configurações de aplicações
- Definições de infraestrutura
- Parâmetros de deploy

O cluster passa a ser uma **consequência** do que está definido no Git.

4.3.2 Mudanças Declarativas e Versionadas

Todas as alterações são feitas de forma declarativa, por meio de arquivos de configuração versionados.

Esse modelo traz benefícios claros:

- Histórico completo de mudanças
 - Auditoria simples e confiável
 - Possibilidade de rollback para estados anteriores
 - Padronização entre ambientes
-

4.3.3 Automação e Conciliação Contínua

Um dos pilares do GitOps é a existência de um **agente automatizado** que compara continuamente:

- Estado desejado (Git)
- Estado real (cluster)

Quando uma divergência é detectada, esse agente executa a reconciliação conforme regras pré-definidas, garantindo aderência ao estado declarado.

4.3.4 Feedback e Observabilidade

GitOps também incentiva ciclos rápidos de feedback. Divergências, falhas de sincronização ou erros de aplicação tornam-se visíveis rapidamente, permitindo ação corretiva antes que impactos maiores ocorram.

4.4 Fluxo Básico do GitOps

Um fluxo típico de GitOps segue os seguintes passos:

1. Um desenvolvedor ou operador altera arquivos no repositório Git
2. A mudança é versionada via commit

3. Um agente GitOps detecta a alteração
4. O estado desejado é aplicado no cluster
5. O sistema monitora e garante conformidade contínua

Esse fluxo elimina deploys manuais e reduz significativamente o risco de inconsistências.

Figura 4 — Fluxo GitOps: Commit → Repositório Git → Reconciliação → Kubernetes

4.5 Benefícios do GitOps

A adoção do GitOps traz vantagens significativas:

- Consistência entre ambientes
- Automação completa do deploy
- Auditoria e rastreabilidade
- Rollback rápido e confiável
- Redução de erros humanos

Esses benefícios tornam-se ainda mais relevantes em ambientes Kubernetes de médio e grande porte.

4.6 GitOps não é CI/CD Tradicional

Embora GitOps se relacione com CI/CD, ele não substitui completamente pipelines de CI. O papel de cada um é distinto:

- **CI**: build, testes e geração de artefatos
- **GitOps (CD)**: aplicação declarativa do estado desejado

No GitOps, o deploy não é acionado por um pipeline empurrando código para o cluster, mas sim por um agente que **observa o Git**.

Essa inversão de controle aumenta a segurança e a previsibilidade do ambiente.

4.7 GitOps e Segurança Operacional

Ao eliminar acessos diretos ao cluster para deploys manuais, o GitOps melhora significativamente a postura de segurança.

Principais ganhos:

- Menos credenciais privilegiadas
- Controle de acesso via Git

- Revisão de mudanças por pull requests
 - Auditoria nativa de todas as alterações
-

Objetivo do Capítulo (Bloco Didático)

Ao final deste capítulo, você deve ser capaz de:

- Explicar o conceito de GitOps
 - Identificar seus princípios fundamentais
 - Compreender o fluxo básico de GitOps
 - Diferenciar GitOps de CI/CD tradicional
-

Exercício de Reflexão (Opcional)

Avalie o seu ambiente atual:

- Onde está definida a configuração dos deploys?
- Existe um repositório que representa o estado desejado?
- Deploys ainda dependem de acesso direto ao cluster?

Essas respostas ajudam a entender o impacto do GitOps no seu contexto.

Checklist de Aprendizado

- Sei definir o que é GitOps
 - Entendo o papel do Git como fonte da verdade
 - Compreendo o fluxo GitOps
 - Consigo diferenciar GitOps de pipelines tradicionais
-

Perguntas de Revisão

1. Qual problema central o GitOps se propõe a resolver?
 2. Por que o Git é considerado a fonte da verdade nesse modelo?
 3. Qual a principal diferença entre GitOps e CD tradicional?
-



Próximo Capítulo

No próximo capítulo, vamos conectar todos esses conceitos ao **ArgoCD**, entendendo como essa ferramenta implementa GitOps na prática em ambientes Kubernetes.

Capítulo 5 — GitOps com ArgoCD

5.1 O Papel do ArgoCD no Modelo GitOps

O GitOps define o **modelo operacional**, mas para que ele funcione na prática é necessário um agente que execute a reconciliação contínua entre o estado desejado e o estado real do cluster. É exatamente esse o papel do **ArgoCD**.

O ArgoCD é uma ferramenta de entrega contínua declarativa para Kubernetes que implementa GitOps de forma nativa. Ele atua como um **controlador** que:

- Observa repositórios Git
- Compara o estado desejado com o estado atual do cluster
- Aplica mudanças de forma controlada
- Exibe visibilidade completa sobre divergências e status

Diferentemente de pipelines tradicionais, o ArgoCD **não empurra código para o cluster**. Em vez disso, ele executa o modelo pull-based, no qual o próprio cluster busca o estado desejado.

5.2 Arquitetura do ArgoCD

A arquitetura do ArgoCD é composta por alguns componentes principais:

- **API Server**: expõe a API e a interface web
- **Repository Server**: acessa e renderiza o conteúdo dos repositórios Git
- **Application Controller**: executa a reconciliação entre Git e cluster
- **Redis**: utilizado para cache e performance

Esses componentes trabalham em conjunto para garantir que o cluster reflita continuamente o estado definido no Git.

Figura 5 — Arquitetura simplificada do ArgoCD

5.3 Conceito de Application

O conceito central do ArgoCD é a **Application**.

Uma Application representa a **ligação entre um repositório Git e um destino Kubernetes**.

Ela define:

- Onde está o código/configuração (repositório, branch, caminho)

- Para qual cluster os recursos serão aplicados
- Em qual namespace o deploy ocorrerá
- Qual política de sincronização será utilizada

Em termos práticos, a Application descreve **o que deve existir no cluster e de onde isso vem**.

5.4 Estrutura Básica de uma Application

Abaixo está um exemplo completo de uma Application declarativa no ArgoCD:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: app-exemplo
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/org/repo.git
    targetRevision: main
    path: k8s/app-exemplo
  destination:
    server: https://kubernetes.default.svc
    namespace: app-exemplo
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

Esse manifesto declara que o ArgoCD deve:

- Observar o repositório Git
 - Ler os manifestos no caminho especificado
 - Aplicá-los no cluster e namespace definidos
 - Garantir aderência contínua ao Git
-

5.5 Sincronização e Estado da Application

Cada Application no ArgoCD possui dois estados principais:

5.5.1 Sync Status

Indica se o estado do cluster está alinhado com o estado definido no Git.

- **Synced**: cluster e Git estão alinhados
 - **OutOfSync**: há divergências entre Git e cluster
-

5.5.2 Health Status

Indica a saúde dos recursos Kubernetes criados pela Application.

- **Healthy**: recursos estão operacionais
- **Progressing**: recursos ainda estão sendo criados
- **Degraded**: há falhas nos recursos

Esses dois indicadores fornecem visibilidade imediata sobre a situação da aplicação.

5.6 Fluxo de Deploy com ArgoCD

Um fluxo típico de deploy utilizando ArgoCD segue os passos abaixo:

1. Um commit é realizado no repositório Git
2. O ArgoCD detecta a alteração
3. A Application entra em estado **OutOfSync**
4. O ArgoCD aplica as mudanças automaticamente ou aguarda ação manual
5. O cluster passa a refletir o novo estado desejado

Esse processo é totalmente auditável e reproduzível.

5.7 ArgoCD e Governança Básica

Mesmo em ambientes simples, o ArgoCD já oferece mecanismos importantes de governança:

- Separação por Applications
- Restrição por Projects
- Controle de destino (cluster/namespace)
- Histórico de sincronizações

Esses mecanismos serão aprofundados nos próximos capítulos, mas já reforçam o modelo GitOps desde o início.



Objetivo do Capítulo (Bloco Didático)

Ao final deste capítulo, você deve ser capaz de:

- Explicar o papel do ArgoCD no GitOps
 - Entender a arquitetura básica do ArgoCD
 - Compreender o conceito de Application
 - Interpretar estados de Sync e Health
-

Exercício Prático (Opcional)

Pense em uma aplicação simples que você conhece:

- Onde ficariam os manifests Kubernetes?
- Qual seria o repositório Git?
- Qual namespace seria utilizado no cluster?

Esse exercício ajuda a consolidar o conceito de Application.

Checklist de Aprendizado

- Sei explicar o que é o ArgoCD
 - Entendo como o ArgoCD implementa GitOps
 - Compreendo o conceito de Application
 - Sei interpretar Sync e Health Status
-

Perguntas de Revisão

1. Qual a principal diferença entre ArgoCD e pipelines tradicionais de CD?
 2. O que uma Application representa no ArgoCD?
 3. Por que o modelo pull-based é mais seguro em Kubernetes?
-

Próximo Capítulo

No próximo capítulo, entraremos na **parte prática**, preparando o ambiente local com Kubernetes e instalando as ferramentas necessárias para acompanhar os exemplos deste ebook.

Capítulo 6 — Preparação do Ambiente

6.1 Visão Geral do Ambiente Utilizado

Ao longo dos capítulos práticos, utilizaremos a seguinte base:

- **Rancher Desktop** para provisionar Kubernetes localmente
- **kubectl** para interação com o cluster
- **Helm** para gerenciamento de pacotes Kubernetes
- **ArgoCD** como ferramenta GitOps

Essa combinação oferece um equilíbrio ideal entre simplicidade, fidelidade ao ambiente real e facilidade de instalação.

6.2 Kubernetes Local com Rancher Desktop

O Rancher Desktop é uma ferramenta gratuita e open source que permite executar Kubernetes localmente em macOS, Windows e Linux. Ele é amplamente utilizado por desenvolvedores e profissionais de plataforma para testes, estudos e desenvolvimento local.

Entre seus principais benefícios estão:

- Kubernetes pronto para uso
- Integração simples com Docker ou containerd
- Interface gráfica intuitiva
- Compatibilidade com kubectl e Helm

Neste ebook, o Rancher Desktop será utilizado como base para todos os exemplos práticos.

Observação

O processo completo de instalação do Rancher Desktop está descrito no **Apêndice A**. Caso ainda não tenha instalado, este é o momento ideal para fazê-lo.

6.3 Verificando o Cluster Kubernetes

Após a instalação do Rancher Desktop, é importante validar se o cluster Kubernetes está ativo e acessível.

No terminal, execute:

```
kubectl cluster-info
```


Se o cluster estiver corretamente configurado, você verá informações indicando o endereço do Kubernetes API Server.

Em seguida, verifique os nós do cluster:

```
kubectl get nodes
```

O estado esperado é **Ready**, indicando que o cluster está operacional.

6.4 Instalando e Validando o Helm

O **Helm** é o gerenciador de pacotes do Kubernetes e será utilizado em diversos exemplos, tanto diretamente quanto por meio do ArgoCD.

Observação

O processo de instalação do Helm está documentado no **Apêndice B**.

Após a instalação, valide com:

```
helm version
```

A saída deve exibir a versão instalada do Helm.

6.5 Preparando o Ambiente para o ArgoCD

Com o cluster Kubernetes ativo e as ferramentas básicas instaladas, o ambiente está pronto para receber o ArgoCD.

Nos próximos capítulos, iremos:

- Instalar o ArgoCD no cluster local
- Acessar a interface web
- Configurar autenticação inicial
- Preparar o ambiente para criar Applications

Essas etapas serão realizadas de forma prática e incremental.

Objetivo do Capítulo (Bloco Didático)

Ao final deste capítulo, você deve ser capaz de:

- Entender a importância da preparação do ambiente
- Identificar as ferramentas utilizadas ao longo do ebook

- Validar o funcionamento do cluster Kubernetes local
 - Garantir que kubectl e Helm estão operacionais
-

Exercício Prático (Opcional)

Antes de avançar, confirme:

- O Rancher Desktop está em execução
- O comando `kubectl get nodes` retorna o cluster como Ready
- O Helm responde corretamente no terminal

Esses passos garantem que você conseguirá acompanhar os próximos capítulos sem impedimentos técnicos.

Checklist de Aprendizado

- Tenho um cluster Kubernetes local funcional
 - Sei validar o acesso ao cluster
 - kubectl está instalado e configurado
 - Helm está instalado e funcionando
-

Perguntas de Revisão

1. Por que utilizar um ambiente Kubernetes local para estudos?
 2. Qual o papel do kubectl nesse contexto?
 3. Em que situações o Helm será utilizado ao longo do ebook?
-

Próximo Capítulo

No próximo capítulo, vamos **instalar e acessar o ArgoCD**, preparando o ambiente para criar nossas primeiras Applications e iniciar a prática de GitOps.

Capítulo 7 — Instalação e Acesso ao ArgoCD

7.1 Visão Geral da Instalação do ArgoCD

Com o ambiente Kubernetes devidamente preparado, é hora de instalar o **ArgoCD** no cluster. Nesta etapa, vamos implantar o ArgoCD de forma **local e controlada**, utilizando manifests oficiais do projeto, garantindo alinhamento com as boas práticas recomendadas.

O objetivo deste capítulo não é apenas executar a instalação, mas também **compreender o que está sendo instalado**, como acessar a interface e validar que o ArgoCD está funcionando corretamente antes de avançar para a criação de Applications.

Observação

Todo o processo descrito aqui é compatível com ambientes macOS, Windows e Linux quando utilizado com Rancher Desktop.

7.2 Criando o Namespace do ArgoCD

Por padrão, o ArgoCD é instalado em um namespace dedicado, chamado **argocd**. Essa separação facilita a organização, governança e futuras manutenções da ferramenta.

Execute o comando abaixo para criar o namespace:

```
kubectl create namespace argocd
```

Caso o namespace já exista, o Kubernetes retornará uma mensagem informando, o que não representa um problema.

7.3 Instalando o ArgoCD no Cluster

A instalação do ArgoCD será feita aplicando o manifesto oficial disponibilizado pelo projeto. Esse manifesto cria todos os recursos necessários, como Deployments, Services, ConfigMaps e Secrets.

Execute o comando:

```
kubectl apply -n argocd \
-f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

Esse comando inicia a criação dos componentes do ArgoCD no namespace **argocd**.

7.4 Verificando a Instalação

Após aplicar o manifesto, é importante verificar se todos os pods foram criados corretamente e estão em execução.

Execute:

```
kubectl get pods -n argocd
```

O estado esperado é que todos os pods estejam como **Running** ou **Completed**. Alguns pods podem levar alguns instantes para iniciar, especialmente em máquinas com menos recursos.

Caso algum pod fique em estado de erro, utilize:

```
kubectl describe pod <nome-do-pod> -n argocd
```

Esse comando ajuda a identificar problemas de configuração ou recursos insuficientes.

7.5 Acessando a Interface Web do ArgoCD

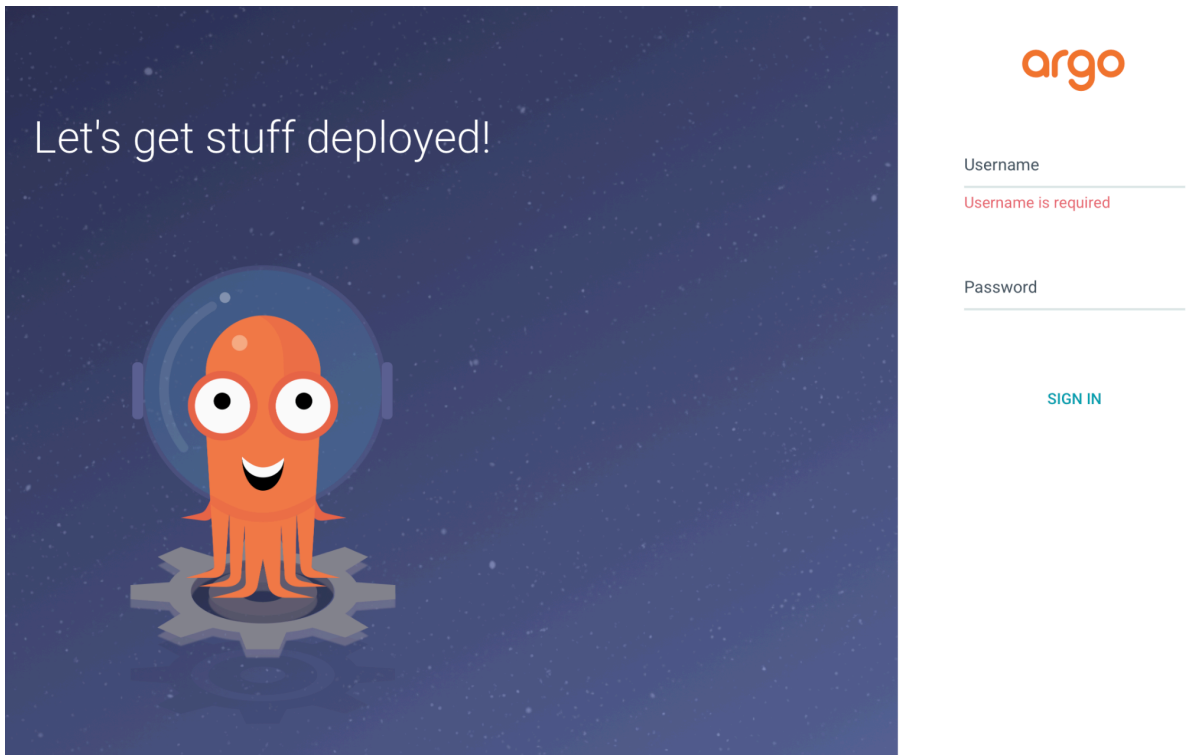
Em ambientes locais, a forma mais simples de acessar a interface web do ArgoCD é utilizando **port-forward**, que cria um túnel seguro entre sua máquina e o serviço do ArgoCD dentro do cluster.

Execute o comando abaixo em um terminal:

```
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

Com o port-forward ativo, acesse no navegador:

```
https://localhost:8080
```



⚠️ Aviso de Segurança

O ArgoCD utiliza um certificado TLS auto-assinado. O navegador exibirá um aviso de segurança, que pode ser ignorado para fins de estudo e desenvolvimento local.

7.6 Obtendo a Senha Inicial do Usuário Admin

Na primeira execução, o ArgoCD cria automaticamente um usuário padrão chamado **admin**, com uma senha armazenada em um Secret do Kubernetes.

Para obter essa senha, execute:

```
kubectl get secret argocd-initial-admin-secret \
-n argocd \
-o jsonpath="{.data.password}" | base64 --decode
```

```
➔ argocd-gitops-na-pratica git:(main) kubectl get secret argocd-initial-admin-secret \
-n argocd \
-o jsonpath="{.data.password}" | base64 --decode
e9h7jL5RkdwpihuF%
```

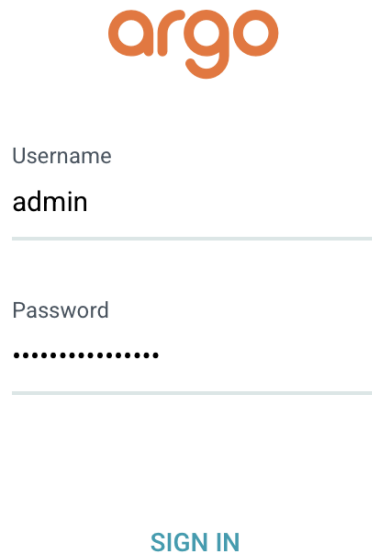
Copie o valor exibido e utilize-o para realizar o login na interface web.

Nota: O símbolo % é apenas um marcador de fim de texto no bash.

7.7 Primeiro Acesso ao ArgoCD

Ao acessar a interface web:

- **Usuário:** `admin`
- **Senha:** valor obtido no passo anterior



The login form features the orange 'argo' logo at the top. Below it, there are two input fields: 'Username' with the text 'admin' and 'Password' with masked characters '.....'. A blue 'SIGN IN' button is positioned at the bottom of the form.

Após o login, você será direcionado para o **Dashboard do ArgoCD**, onde ainda não haverá nenhuma Application criada.

Esse painel centraliza:

- Visualização de Applications
- Status de sincronização
- Saúde dos recursos
- Acesso às configurações

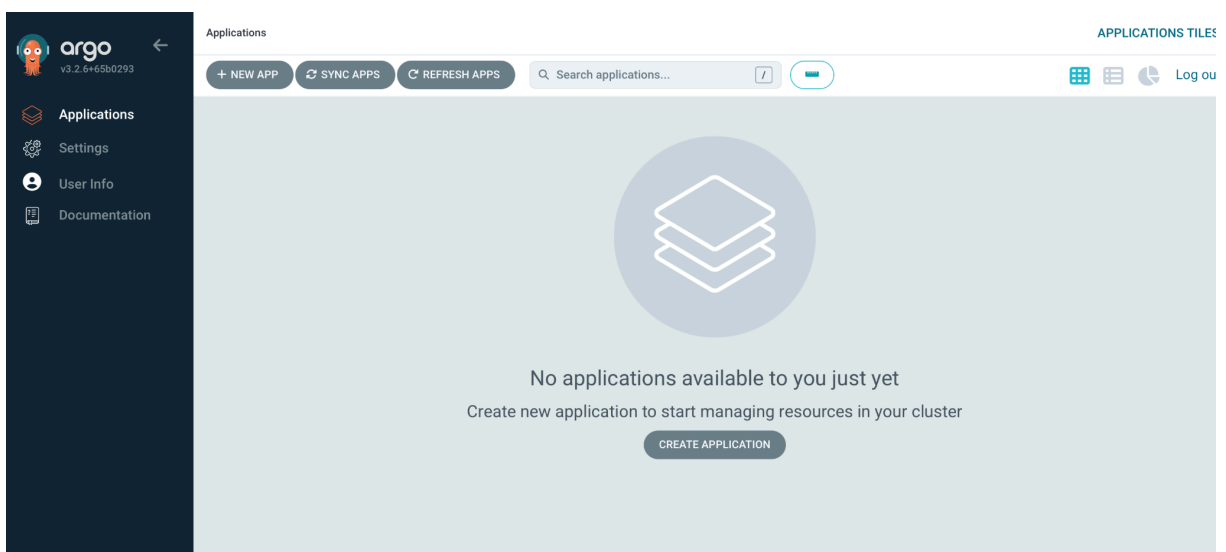


Figura 7 — Dashboard inicial do ArgoCD após o primeiro login

7.8 Considerações Iniciais de Segurança

Mesmo em ambientes de estudo, é importante adotar boas práticas desde o início. Algumas recomendações básicas:

- Alterar a senha do usuário `admin` após o primeiro acesso
- Evitar o uso do usuário `admin` no dia a dia
- Utilizar usuários e permissões específicas conforme a evolução do ambiente

Esses pontos serão aprofundados em capítulos posteriores.

Objetivo do Capítulo (Bloco Didático)

Ao final deste capítulo, você deve ser capaz de:

- Instalar o ArgoCD em um cluster Kubernetes
 - Verificar o estado dos componentes
 - Acessar a interface web do ArgoCD
 - Realizar o primeiro login com segurança
-

Exercício Prático (Opcional)

Antes de avançar, confirme:

- Todos os pods do namespace `argocd` estão em execução
- O Dashboard do ArgoCD está acessível via navegador
- Você consegue realizar login com o usuário `admin`

Esses passos garantem que o ambiente esteja pronto para os próximos capítulos.

Checklist de Aprendizado

- ArgoCD instalado com sucesso
 - Pods em execução no namespace `argocd`
 - Interface web acessível
 - Login realizado com sucesso
-

? Perguntas de Revisão

1. Por que o ArgoCD é instalado em um namespace dedicado?
 2. Qual a vantagem de utilizar port-forward em ambientes locais?
 3. Onde o ArgoCD armazena a senha inicial do usuário admin?
-



Próximo Capítulo

No próximo capítulo, vamos iniciar a **operação prática do ArgoCD**, conhecendo a interface, criando Projects e preparando o ambiente para nossas primeiras Applications.

Capítulo 8 — Uso Básico do ArgoCD

8.1 Visão Geral da Interface do ArgoCD

Após a instalação e o primeiro acesso, o **Dashboard do ArgoCD** se torna o ponto central de operação da plataforma. É por meio dele que você cria Applications, acompanha o estado de sincronização, analisa diferenças entre Git e cluster e executa ações operacionais controladas.

A interface do ArgoCD foi projetada para fornecer **visibilidade imediata** sobre:

- Quantas Applications existem
 - O estado de sincronização (Synced / OutOfSync)
 - A saúde dos recursos (Healthy / Degraded / Progressing)
-

8.2 Navegação Básica

No menu lateral esquerdo, estão as principais áreas do ArgoCD:

- **Applications:** lista e gerenciamento das aplicações
- **Projects:** governança e escopo das Applications
- **Settings:** configurações globais, repositórios e clusters
- **User Info:** informações do usuário logado

Neste capítulo, vamos focar principalmente em **Applications** e **Projects**, que formam a base do uso diário do ArgoCD.

8.3 Projects no ArgoCD

Antes de criar Applications, é importante entender o conceito de **Project** no ArgoCD.

Um Project (tecnicamente chamado de **AppProject**) funciona como uma **camada de governança**, definindo limites claros sobre o que uma Application pode ou não fazer.

Um Project pode restringir:

- Quais repositórios Git podem ser utilizados
- Em quais clusters o deploy é permitido
- Quais namespaces podem ser usados
- Quais tipos de recursos Kubernetes podem ser criados

Mesmo em ambientes simples, o uso de Projects ajuda a evitar erros e prepara o ambiente para escalar com segurança.

8.4 Criando um Project

Para criar um novo Project pela interface web:

1. Acesse **Settings** → **Projects**
2. Clique em **New Project**
3. Preencha os campos básicos:
 - **Name:** `project-estudos`
 - **Description:** Project para estudos com ArgoCD
 - Clique em **Create**
4. Em **Source Repositories**, click em Edit e permita os repositórios desejados
 - Para estudos, pode-se utilizar *
5. Em **Destinations**, click em Edit e configure:
 - Cluster: `https://kubernetes.default.svc`
 - Namespace: *
6. Em **Cluster Resources White List** adicione um recurso com (use valores restritivos em um project real):
 - Kind: *
 - Group: *

Após a criação, o Project já estará disponível para associação às Applications.

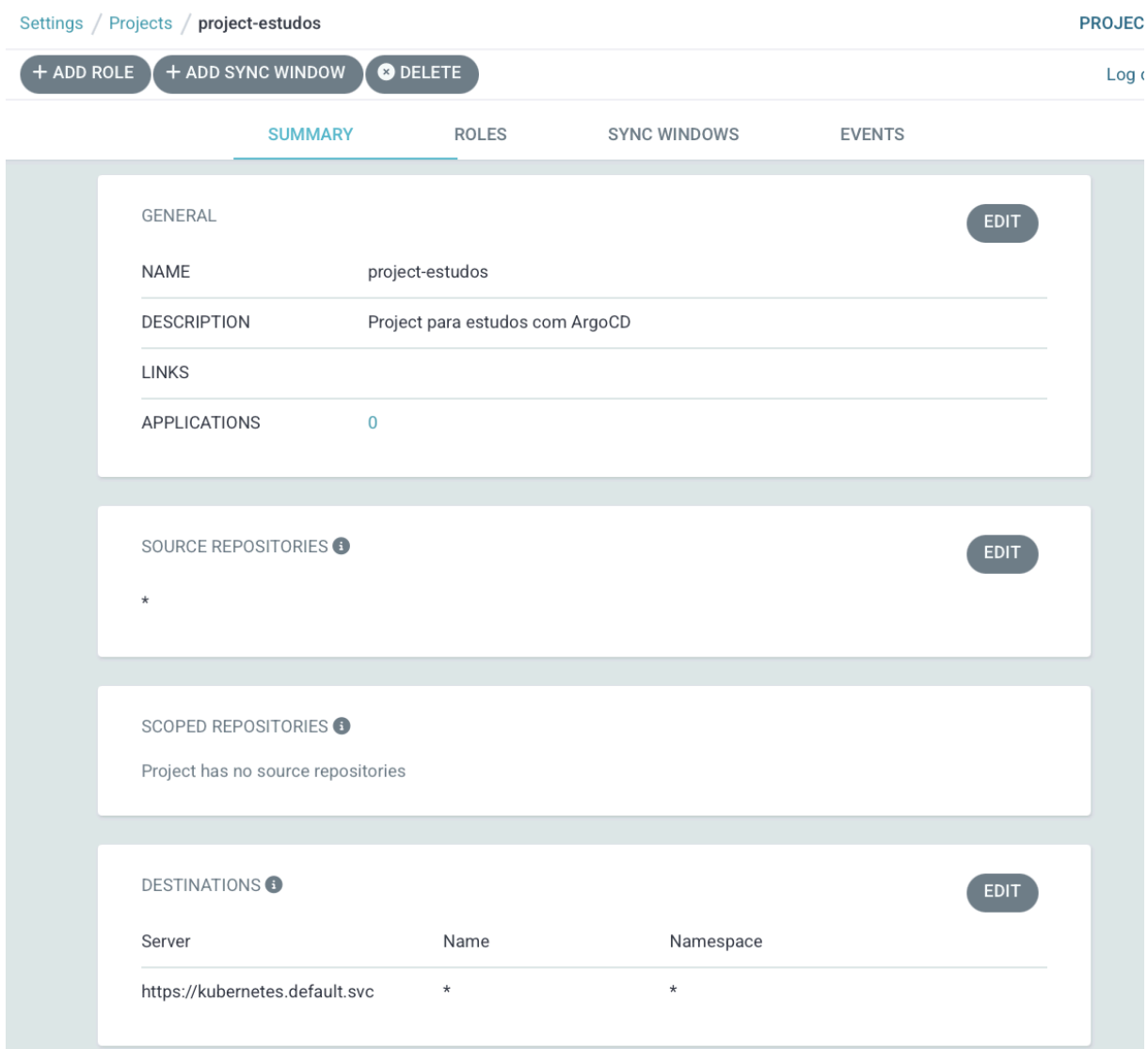


Figura 9 — Criação de um Project no ArgoCD

8.5 Conceito de Application na Prática

Com o Project criado, podemos avançar para o conceito mais importante do ArgoCD: a **Application**.

Uma Application define:

- **Fonte (Source)**: de onde vêm os manifests (Git)
- **Destino (Destination)**: onde os recursos serão aplicados (cluster/namespace)
- **Política de sincronização (Sync Policy)**

Criar uma Application é declarar **como o Git se transforma em recursos Kubernetes**.

8.6 Criando uma Application pela Interface Web

Para criar uma nova Application:

1. Clique em **New App**
2. Preencha a seção **General**:
 - **Application Name**: `app-exemplo`
 - **Project**: `project-estudos`
 - **Sync Policy**: `Manual`
3. Configure a seção **Source**:
 - **Repository URL**: `https://github.com/org/repo.git`
 - **Revision**: `main`
 - **Path**: `k8s/app-exemplo`
4. Configure a seção **Destination**:
 - **Cluster URL**: `https://kubernetes.default.svc`
 - **Namespace**: `app-exemplo`
5. Clique em **Create**

Crie o configmap abaixo na pasta de destino para ter algo pra ver na tela:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-exemplo-config
data:
  APP_NAME: "app-exemplo"
  APP_ENV: "dev"
```

A Application será criada e exibida no Dashboard.

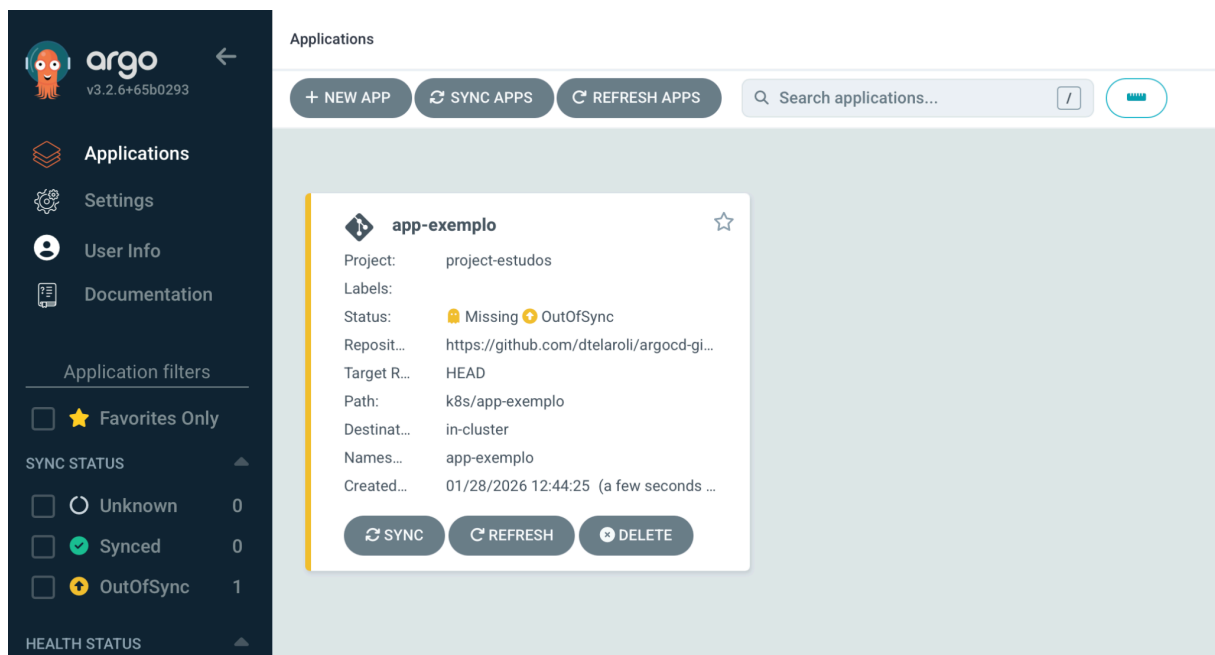
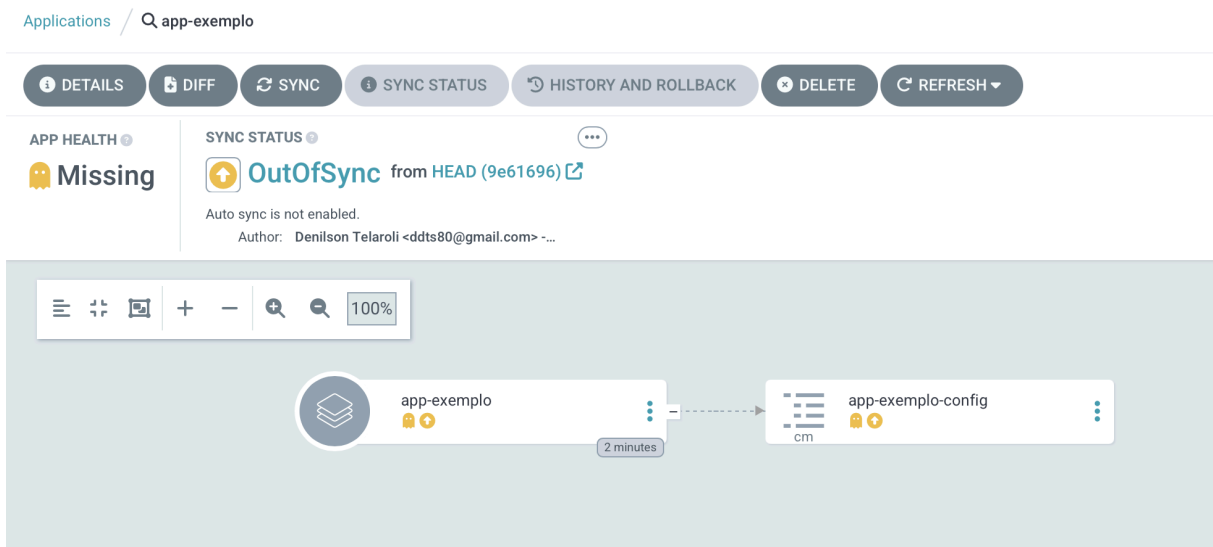


Figura 10 — Tela de criação de uma Application no ArgoCD



8.7 Estados da Application

Após a criação, a Application pode apresentar diferentes estados:

8.7.1 OutOfSync

Indica que os recursos definidos no Git ainda não foram aplicados no cluster ou que existem diferenças entre Git e cluster.

8.7.2 Synced

Indica que o estado do cluster está alinhado com o estado definido no Git.

8.7.3 Healthy / Degraded / Progressing

Esses estados refletem a saúde dos recursos Kubernetes criados, permitindo identificar rapidamente problemas de deploy ou inicialização.

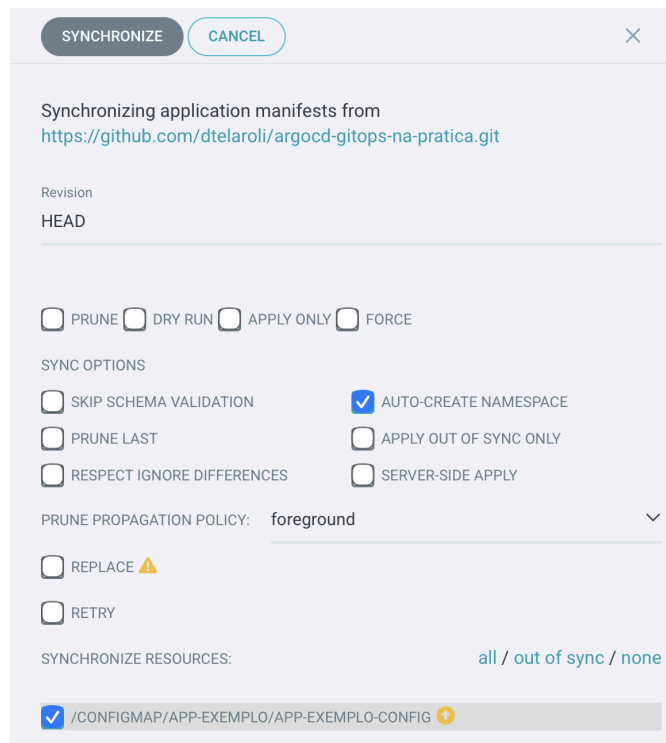
8.8 Sincronizando uma Application Manualmente

Com **Sync Policy Manual**, o ArgoCD detecta divergências, mas não aplica as mudanças automaticamente.

Para sincronizar manualmente:

1. Clique na Application desejada

2. Clique em **Sync**
3. Revise os recursos que serão aplicados
4. Marque a opção **AUTO-CREATE NAMESPACE**
5. Confirme a sincronização clicando no botão **SYNCHRONIZE**



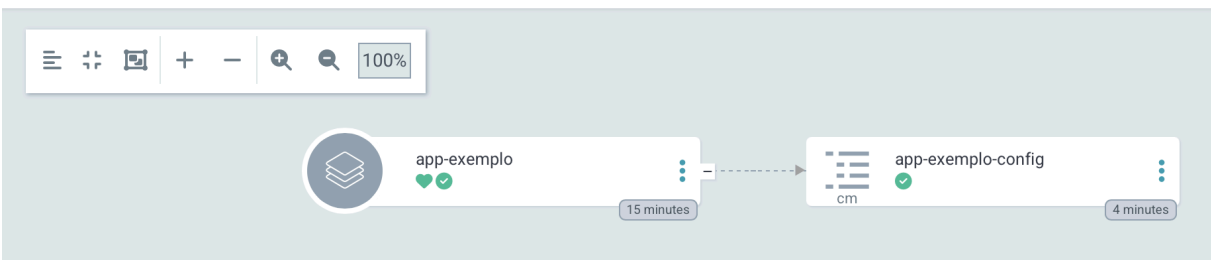
Applications / **app-exemplo**

DETAILS **DIFF** **SYNC** **SYNC STATUS** **HISTORY AND ROLLBACK** **DELETE** **REFRESH**

APP HEALTH
Healthy

SYNC STATUS
Synced to HEAD (9e61696)
Auto sync is not enabled.
Author: Denilson Telaroli <ddts80@gmail.com> -...

LAST SYNC
Sync OK to 9e61696
Succeeded 4 minutes ago (Wed Jan 28 2026 12:55:32 GMT-0300)
Author: Denilson Telaroli <ddts80@gmail.com> -...



Esse modelo é recomendado para ambientes iniciais e críticos, pois oferece maior controle sobre as mudanças.

Em alguns casos, pode ser desejável criar o namespace a priori, para ser possível adicionar configurações extras, como por exemplo: annotations.

Objetivo do Capítulo (Bloco Didático)

Ao final deste capítulo, você deve ser capaz de:

- Navegar pela interface do ArgoCD
 - Criar e entender Projects
 - Criar Applications pela interface web
 - Interpretar estados de Sync e Health
-



Exercício Prático (Opcional)

Crie uma Application simples utilizando um repositório de testes e observe:

- O estado inicial da Application
 - O comportamento ao executar o Sync manual
 - A mudança de OutOfSync para Synced
-



Checklist de Aprendizado

- Sei navegar pela interface do ArgoCD
 - Entendo o papel dos Projects
 - Sei criar uma Application
 - Consigo interpretar estados de Sync e Health
-



Perguntas de Revisão

1. Qual o papel de um Project no ArgoCD?
 2. O que significa uma Application estar OutOfSync?
 3. Quando é indicado utilizar o Sync Manual?
-



Próximo Capítulo

No próximo capítulo, vamos aprofundar o controle de deploys entendendo **Revision e Sync Policy**, conceitos fundamentais para ambientes com múltiplos ambientes e maior governança.

Capítulo 9 — Revision e Sync Policy

9.1 Por Que Revision e Sync Policy São Tão Importantes

Em ambientes Kubernetes gerenciados por GitOps, **controlar o que será implantado e como as mudanças serão aplicadas** é fundamental para garantir previsibilidade, segurança e governança.

No ArgoCD, esses dois controles são exercidos principalmente por meio de:

- **Revision**: define *qual versão* do repositório Git será utilizada
- **Sync Policy**: define *como e quando* as mudanças serão aplicadas no cluster

Uma configuração inadequada desses parâmetros pode resultar em deploys inesperados, inconsistências entre ambientes ou dificuldades de rollback.

9.2 O Conceito de Revision no ArgoCD

A **Revision** determina qual referência do repositório Git o ArgoCD utilizará como fonte da verdade.

Ela pode apontar para:

- Uma **branch**
- Uma **tag**
- Um **commit específico (SHA)**

Esse campo é configurado na Application, geralmente como **targetRevision**.

9.3 Tipos de Revision

9.3.1 Branch

Exemplos comuns:

- **main**
- **develop**
- **feature/nova-funcionalidade**

Características:

- Recebe atualizações contínuas
- Ideal para ambientes de desenvolvimento
- Menos previsível para produção

Uso recomendado:

- Ambientes não críticos
 - Testes e validações rápidas
-

9.3.2 Tag

Exemplos:

- v1.0.0
- v1.2.3

Características:

- Imutável
- Rastreável
- Facilita rollback

Uso recomendado:

- Homologação
- Produção

Tags garantem que o mesmo artefato será utilizado em todos os ambientes, aumentando a confiabilidade do deploy.

9.3.3 Commit (SHA)

Exemplo:

- a1b2c3d4e5f6

Características:

- Controle absoluto sobre o que será implantado
- Extremamente previsível
- Pouco prático para uso contínuo

Uso recomendado:

- Situações específicas
- Auditorias
- Correções emergenciais

9.4 Onde Configurar a Revision

A Revision é definida na Application, no campo `targetRevision`.

Exemplo:

```
source:  
  repoURL: https://github.com/org/repo.git  
  targetRevision: v1.2.0  
  path: k8s/app-exemplo
```

Esse trecho indica que o ArgoCD deve sempre utilizar a tag `v1.2.0` como fonte da verdade.

9.5 Boas Práticas para Revision

Algumas recomendações importantes:

- Utilize **branches** apenas em ambientes de desenvolvimento
- Utilize **tags versionadas** em ambientes críticos
- Evite utilizar `HEAD` em produção
- Versione manifests, Helm charts e overlays junto com a aplicação
- Utilize nomenclatura de tags clara e consistente

Essas práticas facilitam rollback, auditoria e troubleshooting.

9.6 Sync Policy no ArgoCD

A **Sync Policy** define como o ArgoCD aplica as mudanças detectadas no repositório Git no cluster Kubernetes.

Existem dois modos principais:

- **Sync Manual**
 - **Sync Automático (Auto-Sync)**
-

9.7 Sync Manual

No modo **Manual**, o ArgoCD:

- Detecta divergências entre Git e cluster

- Marca a Application como **OutOfSync**
- Aguarda ação explícita do operador para sincronizar

Quando utilizar:

- Ambientes de produção
- Sistemas críticos
- Fluxos que exigem aprovação humana

Esse modelo oferece maior controle e reduz o risco de alterações inesperadas.

9.8 Sync Automático (Auto-Sync)

No **Auto-Sync**, o ArgoCD aplica automaticamente qualquer alteração detectada no repositório configurado.

Exemplo de configuração:

```
syncPolicy:  
  automated:  
    prune: true  
    selfHeal: true
```

9.8.1 Opções do Auto-Sync

Auto-Sync

- Aplica automaticamente mudanças do Git no cluster

Prune

- Remove recursos do cluster que não existem mais no Git

Self Heal

- Reverte alterações manuais feitas diretamente no cluster

Essas opções reforçam o princípio de que **o Git é a única fonte da verdade**.

9.9 Prune e Self Heal em Detalhe

9.9.1 Prune

O **Prune** remove recursos Kubernetes que foram apagados do repositório Git.

Atenção

Em ambientes de produção, o uso de Prune deve ser cuidadosamente avaliado para evitar remoções indesejadas.

9.9.2 Self Heal

O **Self Heal** corrige automaticamente alterações feitas manualmente no cluster, como:

- `kubectl edit`
- `kubectl apply` fora do fluxo GitOps

Esse recurso evita o chamado *configuration drift*.

9.10 Estratégia Recomendada por Ambiente

Uma estratégia comum e eficaz é alinhar **Revision** e **Sync Policy** conforme o ambiente:

Ambiente	Revision	Sync Policy
Desenvolvimento	Branch (<code>develop</code>)	Auto-Sync + Self Heal
Homologação	Tag (<code>vX.Y.Z</code>)	Auto-Sync
Produção	Tag versionada	Sync Manual

Essa abordagem equilibra velocidade, segurança e controle.

Objetivo do Capítulo (Bloco Didático)

Ao final deste capítulo, você deve ser capaz de:

- Entender o conceito de Revision
 - Diferenciar branch, tag e commit
 - Configurar Sync Manual e Auto-Sync
 - Definir estratégias adequadas por ambiente
-

Exercício Prático (Opcional)

Altere a Revision de uma Application existente:

- Teste apontar para uma branch

- Depois altere para uma tag
- Observe o comportamento do ArgoCD

Isso ajuda a visualizar o impacto real dessas configurações.



Checklist de Aprendizado

- Sei configurar Revision corretamente
 - Entendo quando usar branch, tag ou commit
 - Compreendo Sync Manual e Auto-Sync
 - Sei os riscos do Prune em produção
-



Perguntas de Revisão

1. Por que tags são recomendadas em produção?
 2. Qual o risco de utilizar Auto-Sync com Prune sem cuidado?
 3. Como Self Heal reforça o modelo GitOps?
-



Próximo Capítulo

No próximo capítulo, vamos abordar **Projects, Namespaces e Segregação de Ambientes**, entendendo como organizar e proteger ambientes com múltiplos times e aplicações.

Capítulo 10 — Projects, Namespaces e Segregação de Ambientes

10.1 Por Que Segregar Ambientes no ArgoCD

À medida que ambientes Kubernetes crescem e passam a atender múltiplos times, aplicações e estágios (desenvolvimento, homologação e produção), a falta de segregação adequada se torna um risco operacional.

Sem limites claros, surgem problemas como:

- Deploys acidentais em ambientes errados
- Acesso indevido a recursos sensíveis
- Dificuldade de auditoria e governança
- Impactos cruzados entre aplicações

O ArgoCD oferece mecanismos nativos para lidar com esses desafios por meio de **Projects** e da integração com **Namespaces do Kubernetes**.

10.2 Projects (AppProjects) no ArgoCD

Os **Projects**, tecnicamente chamados de **AppProjects**, são uma camada lógica de governança dentro do ArgoCD. Eles definem o **escopo máximo** no qual uma Application pode operar.

Um Project pode restringir:

- Repositórios Git permitidos
- Clusters de destino
- Namespaces autorizados
- Tipos de recursos Kubernetes aceitos
- Ações permitidas (sync, delete, etc.)

Mesmo que uma Application esteja mal configurada, o Project atua como uma **barreira de segurança**, impedindo que limites sejam ultrapassados.

10.3 Benefícios do Uso de Projects

A adoção consistente de Projects traz diversos benefícios:

- **Isolamento entre times e domínios**

- Redução do impacto de erros humanos
- Controle centralizado de permissões
- Facilidade de auditoria e compliance

Projects são especialmente importantes em ambientes corporativos e multi-times.

10.4 Namespaces no Kubernetes

Os **Namespaces** são um recurso nativo do Kubernetes utilizado para isolar recursos dentro de um cluster.

Eles permitem:

- Separar aplicações
- Evitar conflitos de nomes
- Aplicar políticas de segurança
- Controlar consumo de recursos

No contexto do ArgoCD, o namespace representa o **destino final do deploy** de uma Application.

10.5 Boas Práticas para Namespaces

Algumas boas práticas recomendadas:

- Utilizar namespaces dedicados por domínio funcional
- Separar namespaces por ambiente (dev, hml, prd)
- Evitar o uso do namespace **default**
- Adotar um padrão de nomenclatura consistente

Atenção

Evite criar namespaces por aplicação ou por time indiscriminadamente. Com o tempo, aplicações podem ser renomeadas ou reorganizadas, e migrar workloads entre namespaces produtivos não é uma tarefa simples.

Exemplo de nomenclatura por domínio e ambiente:

- vendas-dev
 - vendas-hml
 - vendas-prd
-

10.6 Segregação por Namespace vs Segregação por Cluster

Algumas organizações optam por segregar ambientes utilizando apenas **namespaces dentro de um mesmo cluster**. Essa abordagem é válida e bastante utilizada, pois:

- Reduz custos operacionais
- Simplifica a administração
- Diminui o esforço de manter múltiplos clusters

Entretanto, a **segregação por cluster** é considerada mais segura e robusta, especialmente em ambientes críticos.

Ao isolar ambientes em clusters distintos, elimina-se o risco de:

- Impactos cruzados por consumo excessivo de recursos
- Falhas de configuração afetando outros ambientes
- Incidentes de segurança propagados

Além disso, a segregação por cluster facilita requisitos de **compliance, auditoria e governança**, permitindo políticas específicas por ambiente.

Na prática, muitas organizações adotam um **modelo híbrido**:

- Clusters separados por ambiente
- Namespaces para isolamento lógico dentro do cluster

10.7 Como Projects e Namespaces Trabalham Juntos

Projects e Namespaces atuam de forma complementar:

- **Project** define *onde a Application pode ser implantada*
- **Namespace** define *onde a Application será implantada*

Se uma Application tentar utilizar um namespace não permitido pelo Project, o ArgoCD bloqueará a operação automaticamente.

Esse mecanismo adiciona uma camada extra de segurança e previsibilidade.

10.8 Exemplos de Organização com Projects

Algumas formas comuns de organizar Projects:

Por ambiente

- project-dev
- project-hml
- project-prd

Por time

- project-time-a
- project-time-b

Modelo misto

- project-vendas-dev
- project-vendas-prd

A escolha do modelo depende da maturidade da organização e do nível de isolamento desejado.

10.9 Exemplo Conceitual de Segregação

Um exemplo simples e eficaz:

- **Cluster:** dev
- **Project:** project-vendas
- **Namespace:** vendas-dev
- **Application:** app-carrinho

Esse modelo é fácil de entender, auditar e manter.



Objetivo do Capítulo (Bloco Didático)

Ao final deste capítulo, você deve ser capaz de:

- Entender a importância da segregação de ambientes
 - Diferenciar Projects e Namespaces
 - Escolher uma estratégia adequada de segregação
 - Evitar erros comuns de organização
-



Exercício Prático (Opcional)

Analise o ambiente onde você atua (ou está simulando):

- Quantos ambientes existem?

- Eles estão separados por namespace ou cluster?
- Há Projects definidos no ArgoCD?

Esse exercício ajuda a avaliar o nível de maturidade da governança atual.

Checklist de Aprendizado

- Entendo o papel dos Projects
 - Sei quando usar namespaces
 - Compreendo a diferença entre segregação por cluster e namespace
 - Consigo propor uma estratégia de organização
-

Perguntas de Revisão

1. Qual a principal função de um Project no ArgoCD?
 2. Quais riscos existem ao não segregar ambientes adequadamente?
 3. Quando a segregação por cluster se torna mais indicada?
-

Próximo Capítulo

No próximo capítulo, vamos abordar **ApplicationSet**, entendendo como criar e gerenciar **múltiplas Applications de forma automatizada** no ArgoCD.

Capítulo 11 — ApplicationSet

11.1 O Problema da Escala no ArgoCD

À medida que o uso do ArgoCD cresce, surge um desafio comum: **como gerenciar muitas Applications de forma consistente e escalável**.

Em ambientes reais, é comum encontrar cenários como:

- A mesma aplicação implantada em múltiplos ambientes
- Várias aplicações seguindo exatamente o mesmo padrão
- Criação repetitiva de Applications muito semelhantes
- Alto risco de erro humano por cópia e cola

Criar e manter Applications manualmente nesses cenários não escala bem. É nesse ponto que entra o **ApplicationSet**.

11.2 O Que É ApplicationSet

O **ApplicationSet** é um recurso do ArgoCD que permite **gerar Applications automaticamente** a partir de um template e de uma fonte de dados.

Em vez de criar várias Applications manualmente, você define:

- Um **template** de Application
- Um **gerador** que fornece os parâmetros

O ArgoCD se encarrega de criar, atualizar e remover Applications conforme esses parâmetros mudam.

11.3 Quando Utilizar ApplicationSet

O ApplicationSet é especialmente útil nos seguintes casos:

- Múltiplos ambientes (dev, hml, prd)
- Deploy da mesma aplicação em vários clusters
- Padronização de Applications entre times
- Ambientes multi-tenant
- Redução de esforço operacional

Por outro lado, para cenários simples, com poucas Applications e pouca repetição, o uso de ApplicationSet pode ser desnecessário.

11.4 Componentes do ApplicationSet

Um ApplicationSet é composto basicamente por três partes:

1. **Generators**
 2. **Template**
 3. **Strategy** (opcional)
-

11.4.1 Generators

Os **Generators** são responsáveis por fornecer os dados que serão utilizados para gerar Applications.

Os tipos mais comuns incluem:

- **List Generator**
- **Git Generator**
- **Cluster Generator**
- **Matrix Generator**

Cada gerador atende a um tipo específico de cenário.

11.5 List Generator

O **List Generator** é o mais simples e direto. Ele utiliza uma lista fixa de parâmetros definida no próprio manifesto.

Exemplo:

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: apps-exemplo
spec:
  generators:
    - list:
        elements:
          - env: dev
            namespace: app-dev
          - env: hml
            namespace: app-hml
          - env: prd
            namespace: app-prd
```

```
template:
  metadata:
    name: app-{{env}}
  spec:
    project: project-estudos
    source:
      repoURL: https://github.com/org/repo.git
      targetRevision: main
      path: k8s/app
    destination:
      server: https://kubernetes.default.svc
      namespace: {{namespace}}
```

Esse exemplo gera automaticamente três Applications, uma para cada ambiente.

11.6 Git Generator

O **Git Generator** utiliza informações diretamente de um repositório Git, como diretórios ou arquivos, para gerar Applications dinamicamente.

Esse modelo é muito utilizado quando:

- Cada diretório representa um ambiente
- Cada diretório representa uma aplicação
- A estrutura do repositório define o deploy

Exemplo conceitual de estrutura:

```
apps/
├── app-a/
│   ├── dev/
│   └── prd/
└── app-b/
    ├── dev/
    └── prd/
```

Nesse caso, o ApplicationSet pode percorrer essa estrutura e criar Applications automaticamente.

11.7 Cluster Generator

O **Cluster Generator** cria Applications com base nos clusters registrados no ArgoCD.

É especialmente útil quando:

- A mesma aplicação precisa ser implantada em vários clusters
- Clusters são adicionados ou removidos com frequência

Cada cluster passa a receber uma Application automaticamente, seguindo o mesmo padrão.

11.8 Template de Application

O **template** define o formato final da Application que será criada.

Ele é muito semelhante a uma Application comum, mas permite o uso de variáveis, como:

- `{{env}}`
- `{{name}}`
- `{{server}}`

Essas variáveis são preenchidas a partir do gerador utilizado.

11.9 Benefícios do ApplicationSet

A adoção do ApplicationSet traz benefícios claros:

- **Redução drástica de configuração repetitiva**
- **Padronização de Applications**
- **Menor risco de erro humano**
- **Facilidade de escalar ambientes e clusters**

Além disso, o ApplicationSet reforça ainda mais o modelo declarativo do GitOps.

11.10 Quando Não Utilizar ApplicationSet

Apesar das vantagens, o ApplicationSet não é indicado em todos os cenários.

Evite utilizar ApplicationSet quando:

- Há poucas Applications
- Cada Application é muito diferente das demais
- A complexidade adicional não se justifica

Em ambientes pequenos, Applications simples costumam ser mais claras e fáceis de manter.

Objetivo do Capítulo (Bloco Didático)

Ao final deste capítulo, você deve ser capaz de:

- Explicar o que é ApplicationSet
 - Identificar quando utilizá-lo
 - Entender o papel dos generators
 - Criar Applications de forma automatizada
-

Exercício Prático (Opcional)

Crie um ApplicationSet simples utilizando o **List Generator**:

- Defina pelo menos dois ambientes
 - Observe as Applications sendo criadas automaticamente
 - Teste remover um item da lista e veja o efeito no ArgoCD
-

Checklist de Aprendizado

- Sei explicar o que é ApplicationSet
 - Entendo os principais tipos de generators
 - Consigo identificar cenários de uso
 - Sei quando NÃO usar ApplicationSet
-

Perguntas de Revisão

1. Qual problema o ApplicationSet resolve?
 2. Qual a diferença entre List Generator e Git Generator?
 3. Quando o uso de ApplicationSet pode ser excessivo?
-

Próximo Capítulo

No próximo capítulo, vamos abordar **Sync Window**, entendendo como controlar **quando** o ArgoCD pode ou não aplicar mudanças no cluster.

Capítulo 12 — Sync Window e Controle de Janelas de Deploy

12.1 Por Que Controlar Quando o Deploy Pode Acontecer

Em ambientes GitOps, o ArgoCD observa continuamente o Git e tenta reconciliar o estado desejado com o estado real do cluster. Em ambientes simples, isso é suficiente. Em ambientes reais, porém, **nem todo momento é apropriado para aplicar mudanças**.

Alguns exemplos comuns:

- Horários de pico de uso
- Janelas de fechamento financeiro
- Processos críticos em execução
- Períodos de suporte reduzido
- Mudanças coordenadas entre múltiplos sistemas

O **Sync Window** permite definir **quando o ArgoCD pode ou não executar sincronizações**, trazendo previsibilidade e segurança operacional.

12.2 O Que é Sync Window no ArgoCD

Sync Window é um mecanismo que permite **permitir ou bloquear sincronizações** em janelas de tempo específicas.

Ele atua no nível de **Project**, afetando todas as Applications associadas a ele.

Com Sync Window é possível:

- Bloquear deploys em horários específicos
- Permitir deploys apenas em janelas controladas
- Diferenciar comportamento por ambiente
- Proteger produção contra deploys acidentais

Importante:

Sync Window **não impede commits no Git**, apenas controla quando o ArgoCD pode aplicar as mudanças.

12.3 Tipos de Sync Window

Existem dois tipos principais:

12.3.1 Allow Window

Define **quando sincronizações são permitidas**.

Fora dessa janela, qualquer tentativa de sync é bloqueada.

Uso comum:

- Produção com janelas bem definidas
 - Ambientes altamente regulados
-

12.3.2 Deny Window

Define **quando sincronizações são proibidas**.

Fora da janela de bloqueio, o ArgoCD pode sincronizar normalmente.

Uso comum:

- Bloquear deploys em horários críticos
 - Manter flexibilidade no restante do tempo
-

12.4 Exemplo Prático de Sync Window em Produção

Exemplo de Project com janela de deploy apenas durante a madrugada:

```
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: backend-prd
spec:
  syncWindows:
    - kind: allow
      schedule: "0 1 * * *"
      duration: 2h
      applications:
        - "*"

```

Nesse exemplo:

- Deploys só são permitidos entre **01:00 e 03:00**
 - Fora desse período, qualquer sync é bloqueado
 - Aplica-se a todas as Applications do Project
-

12.5 Exemplo de Bloqueio em Horário Comercial

Bloqueando deploys durante horário de pico:

```
syncWindows:  
  - kind: deny  
    schedule: "0 8 * * 1-5"  
    duration: 10h  
    applications:  
      - "*"
```

Resultado:

- Deploys bloqueados de **08:00 às 18:00**, de segunda a sexta
 - Sincronizações permitidas fora desse horário
-

12.6 Sync Window e Sincronização Manual

Mesmo com Sync Window configurado:

- O botão **Sync** aparece na interface
- Mas a execução é bloqueada se estiver fora da janela

A funcionalidade de Sync Window tem a opção de permitir sync manual, isto serve para que em emergências, seja possível aplicar uma mudança.

Print sugerido

Mensagem do ArgoCD informando que o sync está bloqueado por Sync Window.

Isso evita deploys acidentais mesmo por usuários autorizados.

12.6.6 Opção “Permitir Sync Manual” no Sync Window

Por padrão, o Sync Window bloqueia **todas as sincronizações**, sejam elas automáticas ou manuais. O ArgoCD oferece, porém, a opção “**Permitir sync manual**”, que permite uma exceção controlada a esse comportamento.

Quando essa opção está habilitada:

- Sincronizações automáticas continuam bloqueadas fora da janela
- Sincronizações iniciadas manualmente são permitidas
- A exceção vale tanto para a interface web quanto para a CLI

Essa funcionalidade existe para permitir **ações humanas controladas**, sem liberar automações fora do horário definido.

12.6.7 Uso Prático em Produção

Um cenário comum é:

- Produção com **Deny Window** em horário comercial
- Auto Sync desabilitado
- **Permitir sync manual** habilitado

Nesse modelo, deploys automáticos ficam bloqueados, mas operadores autorizados conseguem executar **hotfixes ou rollbacks emergenciais** sem precisar alterar a janela.

12.6.8 Quando Evitar Essa Opção

Não é recomendada quando:

- O objetivo é bloquear **qualquer tipo de mudança** fora da janela
- Não existe processo de aprovação claro
- O ambiente depende fortemente de automação
- Há risco de ações manuais não auditadas

Nesses casos, o ideal é manter o Sync Window rígido.

12.6.9 Governança e Segurança

A opção **Permitir sync manual** respeita totalmente o **RBAC**.

Apenas usuários com permissão de **sync** conseguem executar a ação.

Ela não quebra o GitOps, apenas adiciona **flexibilidade operacional controlada**, sendo ideal para ambientes que ainda não operam com automação total baseada em métricas.

12.7 Sync Window e Automações

Sync Window também se aplica a:

- CLI
- Pipelines de CI

- Scripts automatizados

Exemplo via CLI:

```
argocd app sync backend-prd
```

Resultado fora da janela:

- Comando falha
- ArgoCD retorna erro informando bloqueio por Sync Window

Isso garante que automações respeitem as mesmas regras dos operadores humanos.

12.8 Sync Window e Maturidade do Time

Times mais maduros tendem a:

- Reduzir dependência de janelas rígidas
- Utilizar métricas e health checks
- Automatizar rollback
- Operar com auto sync mesmo em produção

Já times em evolução normalmente utilizam Sync Window como:

- Camada adicional de proteção
- Instrumento de controle operacional
- Salvaguarda contra erros humanos

Ambos os modelos são válidos, desde que conscientes.

12.9 Boas Práticas no Uso de Sync Window

Algumas recomendações importantes:

- Usar Sync Window principalmente em produção

- Preferir deny window para maior flexibilidade
 - Documentar janelas claramente
 - Evitar janelas excessivamente restritivas
 - Revisar janelas conforme maturidade do time
-

12.10 Erros Comuns

Alguns erros recorrentes:

- Configurar Sync Window global sem necessidade
- Esquecer de revisar janelas antigas
- Bloquear deploys sem comunicação
- Usar Sync Window como substituto de observabilidade

Sync Window é controle, não solução para falta de processo.



Objetivo do Capítulo (Bloco Didático)

Ao final deste capítulo, você deve ser capaz de:

- Entender o que é Sync Window
 - Configurar janelas de allow e deny
 - Controlar deploys por ambiente
 - Proteger produção contra deploys indevidos
-



Exercício Prático (Opcional)

Crie um Project com:

- Deny Window em horário comercial
- Allow Window em horário de manutenção

Tente executar um sync dentro e fora da janela e observe o comportamento do ArgoCD.



Checklist de Aprendizado

Sei o que é Sync Window

Consigo configurar allow e deny windows

Entendo o impacto em automações

Sei quando usar Sync Window

? Perguntas de Revisão

Qual a diferença entre allow e deny window?

Sync Window impede commits no Git?

Como Sync Window protege produção?



Próximo Capítulo

No próximo capítulo, vamos avançar para **Ignore Differences (Avançado)**, entendendo como lidar com mutações automáticas do cluster sem quebrar o GitOps.

Capítulo 13 — Ignore Differences (Uso Avançado)

13.1 Por Que Ignore Differences é Essencial em Ambientes Reais

Um dos pilares do GitOps é a reconciliação contínua entre o estado desejado definido no Git e o estado real do cluster Kubernetes. Na teoria, qualquer diferença entre esses dois estados deve ser corrigida automaticamente pelo ArgoCD.

Na prática, porém, **nem toda diferença representa um problema real**.

Em ambientes Kubernetes modernos, diversos componentes modificam recursos automaticamente após a aplicação inicial, como:

- Controllers nativos do Kubernetes
- Admission Controllers
- Operators
- Service Meshes
- Ferramentas de segurança e observabilidade

Sem um controle adequado dessas diferenças, o ArgoCD pode marcar Applications constantemente como **OutOfSync**, gerar ruído operacional e até entrar em ciclos desnecessários de reconciliação.

13.2 O Problema do Drift “Esperado”

Nem todo drift é um erro. Em muitos casos, ele é **intencional e esperado**.

Exemplos comuns:

- HPA ajustando o número de réplicas
- Istio injetando sidecars automaticamente
- Controllers adicionando annotations e labels
- Campos preenchidos dinamicamente pelo cluster
- Processos de deploy, como blue-green e canary, que alteram parâmetros temporariamente.

O desafio não é eliminar o drift, mas **ensinar o ArgoCD a ignorar apenas o que não é relevante**, sem esconder problemas reais.

É importante destacar que, ao configurar um **ignoreDifferences**, o campo ignorado **deixa de ser gerenciado pelo ArgoCD**. Isso significa que, caso esse campo seja alterado

manualmente ou por alguma automação externa, o ArgoCD **não irá detectar o drift nem reverter a mudança** durante uma sincronização.

Por esse motivo, essa configuração deve ser aplicada com **muito critério**, evitando sempre campos críticos para o funcionamento da aplicação.

Um exemplo comum ocorre em cenários de **canary deployment utilizando Argo Rollouts**. Durante o processo de deploy, é normal que as rotas do service mesh sejam alteradas dinamicamente, o que pode fazer com que o ArgoCD identifique a aplicação como **OutOfSync**, mesmo sendo um comportamento esperado e temporário. Ao final do rollout, essas alterações desaparecem naturalmente.

Nesses casos, adicionar essas rotas como **ignoreDifferences** ajuda a reduzir ruído operacional e evita falsas indicações de drift. À primeira vista, essa abordagem pode parecer incorreta, mas ela resolve um problema real do processo de deploy.

No entanto, o risco está no efeito colateral: se essas mesmas rotas forem alteradas manualmente ou por alguma automação fora do controle do ArgoCD, **a mudança não será mais visível na ferramenta**, nem será revertida em um novo sync. Em ambientes produtivos, isso pode mascarar problemas, dificultar a detecção de incidentes e prolongar o tempo de diagnóstico.

Portanto, o uso de **ignoreDifferences** deve sempre equilibrar redução de ruído e preservação da visibilidade operacional, garantindo que apenas campos realmente transitórios ou não críticos sejam ignorados.

13.3 Ignore Differences no ArgoCD

O ArgoCD permite definir regras de **ignoreDifferences**, que instruem a ferramenta a desconsiderar diferenças específicas ao comparar o Git com o cluster.

Essas regras podem ser configuradas:

- Diretamente na Application
- Em nível de Project
- Por tipo de recurso
- Por campo específico

O objetivo é reduzir ruído sem comprometer governança.

13.4 Exemplo Básico: Ignorando Réplicas

Um dos exemplos mais comuns é ignorar o campo **replicas** em Deployments quando se utiliza HPA.

```
ignoreDifferences:
  - group: apps
    kind: Deployment
    jsonPointers:
      - /spec/replicas
```

Nesse cenário:

- O HPA pode ajustar réplicas livremente
- O ArgoCD não tenta “corrigir” o valor
- A Application permanece **Synced**

13.5 Ignorando Annotations Geradas Automaticamente

Algumas ferramentas adicionam annotations dinâmicas aos recursos.

Exemplo usando `jqPathExpressions`:

```
ignoreDifferences:
  - kind: Deployment
    jqPathExpressions:
      - .metadata.annotations["sidecar.istio.io/status"]
```

Esse tipo de configuração é muito comum em clusters com Istio ou outros service meshes.

13.6 Ignore Differences em CRDs e Operators

CRDs frequentemente possuem campos de `status` e outros atributos mutáveis que **não devem ser gerenciados pelo GitOps**.

Exemplo ignorando o campo `status`:

```
ignoreDifferences:
  - kind: CustomResourceDefinition
    jsonPointers:
      - /status
```

Isso evita que o ArgoCD tente reconciliar campos que são exclusivamente responsabilidade do controller.

13.7 Ignore Differences em Nível de Project

Em ambientes grandes, repetir regras em todas as Applications não escala bem. Nesse caso, é possível definir regras de ignoreDifferences diretamente no **Project**, aplicando-as a todas as Applications associadas.

Esse modelo é indicado quando:

- O padrão se repete em várias aplicações
 - O comportamento é conhecido e controlado
 - Há governança clara sobre o que pode ser ignorado
-

13.8 Boas Práticas no Uso de Ignore Differences

Algumas práticas importantes:

- Ignorar apenas campos específicos
- Evitar ignorar recursos inteiros
- Documentar cada regra aplicada
- Revisar regras periodicamente
- Testar impactos antes de aplicar em produção

Ignore Differences deve reduzir ruído, não esconder problemas.

13.9 Anti-Padrões Comuns

Alguns erros frequentes:

- Ignorar campos críticos de **spec**
- Ignorar múltiplos recursos sem critério
- Usar ignoreDifferences para mascarar falhas
- Aplicar regras globais sem revisão

Esses anti-padrões enfraquecem o modelo GitOps.

13.10 Ignore Differences e Maturidade do Time

Times mais maduros:

- Usam ignoreDifferences de forma mínima
- Confiam em observabilidade e métricas

- Preferem automação consciente

Times em evolução:

- Precisam de ignoreDifferences para reduzir ruído
- Evitam fadiga operacional
- Evoluem gradualmente o modelo

Ambos os cenários são válidos quando bem controlados.

Objetivo do Capítulo

Ao final deste capítulo, você deve ser capaz de:

- Entender quando diferenças são esperadas
 - Configurar ignoreDifferences corretamente
 - Reduzir ruído operacional sem perder controle
 - Evitar anti-padrões comuns
-



Próximo Capítulo

No próximo capítulo, vamos avançar para **Gestão de Acesso e RBAC no ArgoCD**, para aplicação governança.

Capítulo 14 — Gestão de Acesso e RBAC no ArgoCD

14.1 Por Que Governança é Crítica em Ambientes GitOps

À medida que o ArgoCD passa a ser utilizado por múltiplos times, aplicações e ambientes, a gestão de acesso deixa de ser um detalhe operacional e passa a ser um **pilar de segurança e governança**.

Em ambientes sem controle adequado, é comum surgirem problemas como:

- Deploys acidentais em produção
- Alterações realizadas por usuários não autorizados
- Falta de rastreabilidade sobre quem executou determinada ação
- Uso excessivo do usuário administrador

O ArgoCD endereça esses riscos por meio de **RBAC (Role-Based Access Control)** e do uso de **Projects**, permitindo controlar com precisão **quem pode fazer o quê, onde e como**.

14.2 Conceito de RBAC no ArgoCD

O RBAC do ArgoCD define permissões com base em **papéis (roles)** atribuídos a usuários ou grupos. Essas permissões controlam ações como:

- Visualizar Applications
- Executar sincronizações
- Realizar rollbacks
- Criar ou remover recursos
- Administrar configurações globais

Diferente do RBAC do Kubernetes, o RBAC do ArgoCD atua **no nível da ferramenta**, controlando operações GitOps independentemente das permissões diretas no cluster.

14.3 O Papel dos Projects na Governança

Os **Projects (AppProjects)** são a primeira camada de governança no ArgoCD. Eles definem o **escopo máximo** no qual uma Application pode operar.

Um Project pode restringir:

- Repositórios Git permitidos
- Clusters de destino
- Namespaces autorizados
- Tipos de recursos Kubernetes
- Ações permitidas (sync, delete, etc.)

Mesmo que um usuário tenha permissões amplas, o Project atua como uma barreira adicional, evitando impactos fora do escopo definido.

14.4 Modelo de Acesso por Time e Ambiente

Um padrão comum em ambientes corporativos é combinar **Projects + RBAC** para isolar times e ambientes.

Exemplo conceitual:

- Project: `project-backend-prd`
- Namespace: `backend-prd`
- Usuários autorizados:
 - Time de backend (sync manual)
 - Time de plataforma (admin)

Esse modelo reduz o risco de impacto cruzado entre times e aplicações.

14.5 Definindo Permissões com RBAC

As permissões no ArgoCD são definidas por meio de **políticas declarativas**, geralmente configuradas em um ConfigMap.

Exemplo conceitual de política:

```
p, role:backend-dev, applications, sync, project-backend-dev/*,  
allow  
p, role:backend-dev, applications, get, project-backend-dev/*, allow
```

Nesse exemplo, o papel `backend-dev` pode visualizar e sincronizar Applications do projeto de desenvolvimento, mas não possui permissões fora desse escopo.

14.6 RBAC e Integração com Identidade (SSO)

Em ambientes reais, o ArgoCD costuma ser integrado a um provedor de identidade, como:

- LDAP
- Active Directory
- GitHub
- GitLab
- OIDC

Nesse modelo:

- Usuários são autenticados externamente
- Grupos do provedor de identidade são mapeados para roles do ArgoCD
- A gestão de acesso torna-se centralizada e auditável

Isso evita a criação manual de usuários e reduz erros operacionais.

14.7 Separação entre Usuário Admin e Operação Diária

Uma boa prática essencial é **evitar o uso do usuário admin no dia a dia**.

O usuário admin deve ser utilizado apenas para:

- Configuração inicial
- Integrações de identidade
- Manutenções específicas

A operação diária deve ser feita por usuários com permissões limitadas, alinhadas ao papel de cada time.

14.8 RBAC e Automação

O RBAC também se aplica às automações que utilizam a ArgoCD CLI ou API.

Isso significa que:

- Scripts e pipelines devem utilizar usuários ou tokens dedicados
- Cada automação deve ter apenas as permissões necessárias
- Ações executadas automaticamente são auditáveis

Esse modelo reforça o princípio de **menor privilégio**, essencial em ambientes automatizados.

14.9 Erros Comuns na Gestão de Acesso

Alguns erros recorrentes devem ser evitados:

- Uso compartilhado do usuário admin
- Permissões amplas sem necessidade
- Falta de separação entre ambientes
- Ausência de auditoria

Esses problemas geralmente só aparecem quando o ambiente cresce, tornando a correção mais custosa.

Objetivo do Capítulo (Bloco Didático)

Ao final deste capítulo, você deve ser capaz de:

- Entender a importância do RBAC no ArgoCD
 - Compreender o papel dos Projects na governança
 - Definir modelos de acesso por time e ambiente
 - Aplicar o princípio de menor privilégio
-

Exercício de Reflexão (Opcional)

Reflita sobre seu ambiente atual:

- Quem pode executar deploys hoje?
- Há separação clara entre times e ambientes?
- Existe auditoria sobre ações realizadas?

Essas respostas ajudam a identificar riscos ocultos.

Checklist de Aprendizado

Entendo o papel do RBAC no ArgoCD
Sei como Projects ajudam na governança
Compreendo a integração com identidade
Consigo evitar erros comuns de permissão

? Perguntas de Revisão

Por que RBAC é essencial em ambientes GitOps?

Qual a diferença entre RBAC do ArgoCD e do Kubernetes?

Por que o usuário admin não deve ser usado no dia a dia?



Próximo Volume

No próximo volume (II), vamos avançar questões avançadas.

Conclusão

Este ebook apresentou uma jornada completa desde os fundamentos de DevOps e microsserviços até a adoção prática do GitOps com ArgoCD em ambientes Kubernetes.

Ao longo dos capítulos, ficou claro que o ArgoCD não é apenas uma ferramenta de deploy, mas um componente central de um modelo operacional moderno, declarativo e auditável. O uso do Git como fonte única da verdade, aliado à reconciliação contínua, traz ganhos reais de previsibilidade, segurança e escalabilidade.

Os conceitos explorados como: Applications, Projects, ApplicationSet, Revision, Sync Policy, Sync Window e Ignore Differences, formam uma base sólida para operar Kubernetes de forma profissional.

A partir deste ponto, o leitor está preparado para avançar para temas mais complexos, como pipelines de CI/CD integrados ao GitOps, estratégias avançadas de deploy, segurança, observabilidade e operação em larga escala.

Mais do que aprender comandos ou configurações, o principal objetivo foi transmitir um modelo mental de operação confiável, repetível e alinhado às melhores práticas atuais.

Referências

- Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. **Communications of the ACM**, 59(5), 50-57.
- Calcote, L. (2018). **Istio: Up and Running**. O'Reilly Media.
- Dragoni, N., Giallorenzo, S., Lafuente, A. L., Mazzara, M., Montesi, F., Mustafin, R., & Safina, L. (2017). Microservices: Yesterday, Today, and Tomorrow. In **Present and Ulterior Software Engineering** (pp. 195-216). Springer, Cham.
- Merkel, D. (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. **Linux Journal**, 2014(239), 2.
- Newman, S. (2015). **Building Microservices: Designing Fine-Grained Systems**. O'Reilly Media.
- Richardson, C. (2018). **Microservices Patterns: With examples in Java**. Manning Publications.
- ArgoCD. (2024). ArgoCD Documentation. Retrieved from [ArgoCD Documentation](<https://argo-cd.readthedocs.io/en/stable/>)
- Flux. (2024). Flux Documentation. Retrieved from [Flux Documentation](<https://fluxcd.io/docs/>)
- Jenkins X. (2024). Jenkins X Documentation. Retrieved from [Jenkins X Documentation](<https://jenkins-x.io/docs/>)
- Weaveworks. (2020). GitOps: The Way to Manage Kubernetes. Retrieved from [Weaveworks GitOps](<https://www.weave.works/technologies/gitops/>)

Apêndice A — Instalação do Rancher Desktop (macOS e Windows)

A.1 O Que é o Rancher Desktop

O Rancher Desktop é uma ferramenta gratuita e open source que permite executar um cluster Kubernetes localmente de forma simples e confiável. Ele foi desenvolvido pela SUSE como uma alternativa moderna a soluções como Minikube e Docker Desktop, sendo amplamente utilizado para estudos, desenvolvimento local e testes de plataformas Kubernetes.

Neste ebook, o Rancher Desktop é utilizado como base para todos os exemplos práticos, pois oferece:

- Kubernetes local pronto para uso
 - Integração nativa com **kubectl** e **helm**
 - Suporte a macOS, Windows e Linux
 - Interface gráfica simples e estável
-

A.2 Requisitos Mínimos

Antes da instalação, verifique os requisitos básicos:

macOS

- macOS 11 (Big Sur) ou superior
- Processador Intel ou Apple Silicon (M1/M2/M3)
- Virtualização habilitada
- Pelo menos 8 GB de RAM (recomendado)

Windows

- Windows 10 ou Windows 11 (64 bits)
 - WSL2 habilitado
 - Virtualização ativada na BIOS
 - Pelo menos 8 GB de RAM (recomendado)
-

A.3 Instalação no macOS

A.3.1 Download

1. Acesse o site oficial do projeto:
<https://rancherdesktop.io/>
2. Clique em **Download** e selecione a versão para **macOS**

- Escolha a versão correta conforme sua arquitetura:
 - Apple Silicon (ARM64)
 - Intel (x86_64)
-

A.3.2 Instalação

1. Abra o arquivo `.dmg` baixado
2. Arraste o **Rancher Desktop** para a pasta **Applications**
3. Abra o Rancher Desktop pela primeira vez

Outra opção é instalar via brew:

```
brew install --cask rancher
```

Na primeira execução, o macOS pode exibir um aviso de segurança. Caso isso ocorra:

- Vá em **Preferências do Sistema** → **Privacidade e Segurança**
 - Permita a execução do aplicativo
-

A.3.3 Configuração Inicial

Ao iniciar o Rancher Desktop pela primeira vez:

1. Selecione **Kubernetes** como ambiente de trabalho
2. Escolha o runtime de containers:
 - `containerd` (recomendado)
3. Selecione a versão do Kubernetes (pode manter a padrão)
4. Confirme as configurações e aguarde a inicialização

O processo pode levar alguns minutos na primeira execução.

A.3.4 Validação no macOS

Após a inicialização:

No terminal, execute:

```
kubectl version --client
```

E depois:

```
kubectl get nodes
```

O resultado esperado é um node em estado **Ready**, indicando que o cluster está funcional.

A.4 Instalação no Windows

A.4.1 Pré-requisito: WSL2

O Rancher Desktop no Windows utiliza o **WSL2**.

Caso ainda não esteja habilitado, execute no PowerShell como Administrador:

```
wsl --install
```

Após a instalação, reinicie o computador.

A.4.2 Download

1. Acesse:
<https://rancherdesktop.io/>
 2. Clique em **Download**
 3. Baixe o instalador para **Windows (.exe)**
-

A.4.3 Instalação

1. Execute o arquivo **.exe** baixado
2. Siga o assistente de instalação padrão
3. Conclua a instalação e abra o Rancher Desktop

Na primeira execução, o Rancher Desktop pode solicitar permissões para integração com o WSL2.

A.4.4 Configuração Inicial no Windows

Na tela inicial do Rancher Desktop:

1. Selecione **Kubernetes**
 2. Escolha o runtime:
 - **containerd** (recomendado)
 3. Confirme o uso do WSL2
 4. Mantenha a versão padrão do Kubernetes
 5. Aguarde a inicialização do cluster
-

A.4.5 Validação no Windows

Abra o **PowerShell** ou **Windows Terminal** e execute:

```
kubectl version --client
```

Em seguida:

```
kubectl get nodes
```

Assim como no macOS, o node deve aparecer com status **Ready**.

A.5 Considerações Importantes

- O Rancher Desktop já configura automaticamente o `kubectl`
- Não é necessário instalar Minikube ou Kind
- O cluster local é suficiente para todos os exemplos deste ebook
- Para ambientes de produção, o Rancher Desktop **não deve ser utilizado**

Este ambiente é exclusivamente voltado para **estudo, testes e desenvolvimento local**.

Apêndice B — Instalação do Helm (macOS e Windows)

B.1 O Que é o Helm

O Helm é o gerenciador de pacotes do Kubernetes. Ele permite empacotar, versionar, configurar e implantar aplicações Kubernetes de forma padronizada e reutilizável, por meio dos chamados **Helm Charts**.

No contexto deste ebook, o Helm será utilizado em conjunto com o ArgoCD para:

- Gerenciar aplicações Kubernetes complexas
- Padronizar deploys entre ambientes
- Facilitar versionamento e rollback
- Demonstrar integração nativa com GitOps

O Helm é amplamente adotado no ecossistema Kubernetes e é considerado uma ferramenta essencial para ambientes profissionais.

B.2 Requisitos Mínimos

Antes de instalar o Helm, verifique:

- Cluster Kubernetes funcional
- **kubectl** instalado e configurado
- Acesso ao terminal (macOS ou Windows)

Observação

O Helm não executa containers nem cria clusters. Ele atua **sobre um cluster Kubernetes já existente**.

B.3 Instalação do Helm no macOS

B.3.1 Instalação via Homebrew (Recomendado)

A forma mais simples e recomendada de instalar o Helm no macOS é utilizando o **Homebrew**.

No terminal, execute:

```
brew install helm
```

O Homebrew cuidará automaticamente do download, instalação e atualização do Helm.

B.3.2 Validação da Instalação no macOS

Após a instalação, valide executando:

```
helm version
```

A saída esperada deve exibir a versão instalada do Helm, indicando que o binário está corretamente configurado no sistema.

B.3.3 Verificando Integração com o Kubernetes

Para confirmar que o Helm consegue se comunicar com o cluster Kubernetes:

```
kubectl get nodes
```

Se o cluster estiver acessível e em estado **Ready**, o Helm estará apto para uso nos próximos capítulos.

B.4 Instalação do Helm no Windows

B.4.1 Instalação via Chocolatey (Recomendado)

Se você utiliza o **Chocolatey**, a instalação do Helm é simples.

No PowerShell (como Administrador), execute:

```
choco install kubernetes-helm -y
```

Após a conclusão, feche e reabra o terminal para garantir que o Helm esteja disponível no PATH.

B.4.2 Instalação Manual (Alternativa)

Caso não utilize o Chocolatey:

1. Acesse o site oficial do Helm:
<https://helm.sh/>
 2. Vá até a seção **Get Helm**
 3. Baixe o arquivo **.zip** correspondente ao Windows
 4. Extraia o binário **helm.exe**
 5. Adicione o diretório do **helm.exe** ao PATH do sistema
-

B.4.3 Validação da Instalação no Windows

No PowerShell ou Windows Terminal, execute:

```
helm version
```

A exibição da versão confirma que o Helm está corretamente instalado.

B.5 Conceitos Básicos do Helm

Antes de avançar, é importante entender alguns conceitos fundamentais:

- **Chart**
Pacote Helm que contém templates Kubernetes, valores padrão e metadados.
- **Release**
Uma instância de um Chart instalada em um cluster.
- **Values**
Arquivo ou parâmetros que personalizam o comportamento de um Chart.

Esses conceitos serão explorados na prática ao longo do ebook.

B.6 Considerações Importantes

- O Helm não substitui manifests Kubernetes, ele os abstrai
- Helm Charts devem ser versionados em Git
- O ArgoCD possui suporte nativo a Helm
- O uso de Helm facilita rollback e padronização entre ambientes

Atenção

Mesmo utilizando Helm, o Git deve continuar sendo a **fonte única da verdade** no modelo GitOps.